



注意力机制



归纳



- 识别: 如何判断哪些信息与任务相关/无关? —————→ 打分函数
- 过滤: 如何提取任务相关信息? 忽略任务无关信息? —————→ 提取机制
- 建模: 如何根据提取的任务相关信息实现建模? —————→ 聚合机制

	软性注意力机制	硬性注意力机制	键值对注意力机制	多头注意力机制	
打分函数	$s(x, q)$	$I[s(x, q); \lambda]$	$s(k, q)$	$s(k, q_i)$	
信息提取机制	$s(x, q) \cdot x$	argmax	$s(k, q) \cdot v$	$s(k, q_i) \cdot v$	
聚合机制	加和	argmax	加和	拼接, 加和	

- 如何表示任务相关这一点呢? Query $\rightarrow q$

注意力打分函数 $s(\mathbf{x}_n, \mathbf{q})$



加性模型

$$s(\mathbf{x}, \mathbf{q}) = \mathbf{v}^\top \tanh(\mathbf{W}\mathbf{x} + \mathbf{U}\mathbf{q}),$$

点积模型

$$s(\mathbf{x}, \mathbf{q}) = \mathbf{x}^\top \mathbf{q},$$

缩放点积模型

$$s(\mathbf{x}, \mathbf{q}) = \frac{\mathbf{x}^\top \mathbf{q}}{\sqrt{D}},$$

双线性模型

$$s(\mathbf{x}, \mathbf{q}) = \mathbf{x}^\top \mathbf{W} \mathbf{q},$$

任务相关性越大，分值越大

注意力模型



软性注意力机制 (soft attention mechanism)

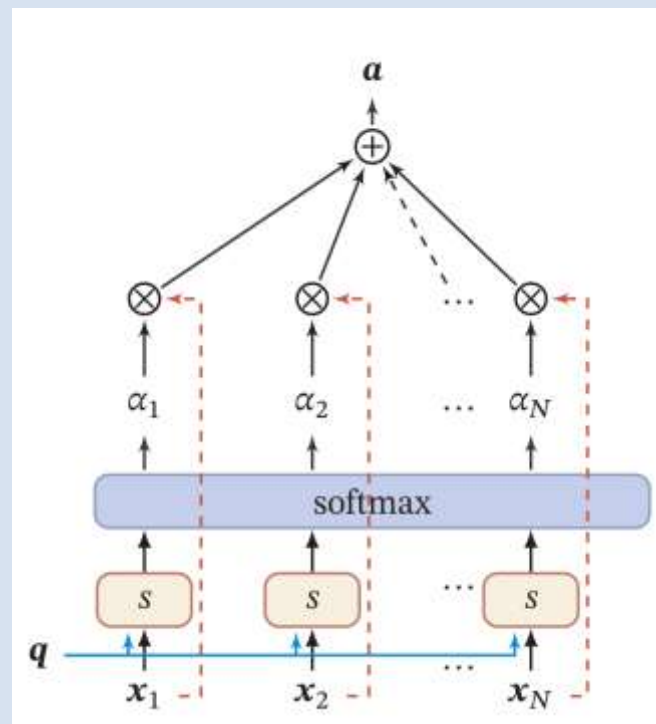
注意力机制可以分为两步：

Step1 计算注意力分布 α

$$\begin{aligned}\alpha_n &= p(z = n | X, q) \\ &= \text{softmax}(s(x_n, q)) \\ &= \frac{\exp(s(x_n, q))}{\sum_{j=1}^N \exp(s(x_j, q))}\end{aligned}$$

Step2 根据 α 来计算输入信息的加权平均

$$\begin{aligned}\text{att}(X, q) &= \sum_{n=1}^N \alpha_n x_n, \\ &= \mathbb{E}_{z \sim p(z | X, q)}[x_z]\end{aligned}$$



注意力机制的变体



多头注意力（multi-head attention）

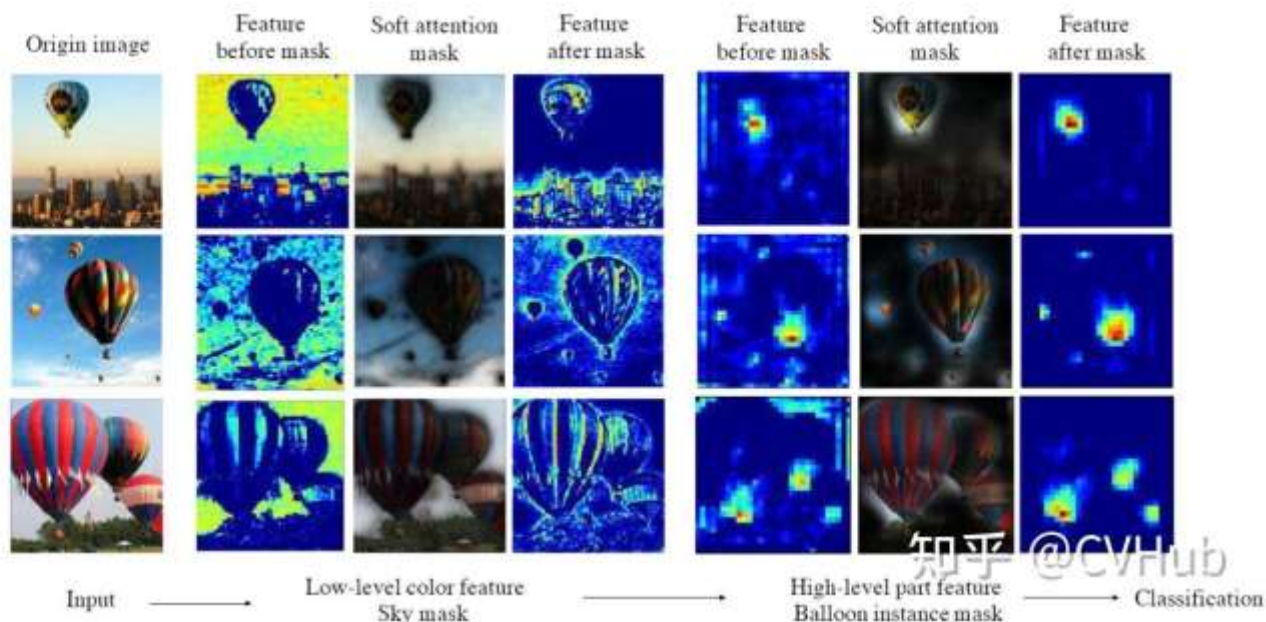
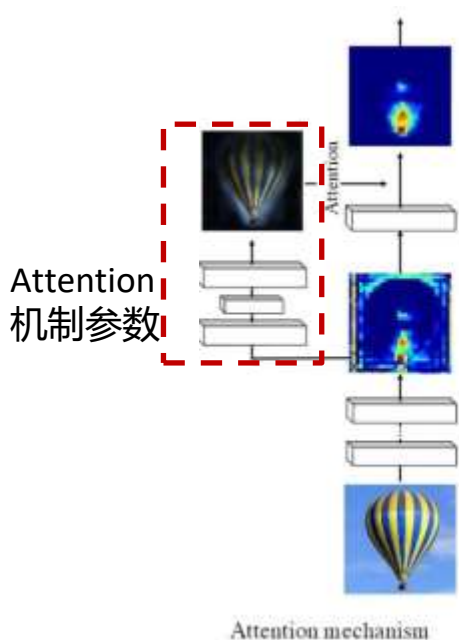
利用多个查询 $Q=[q_1, \dots, q_M]$ ，来并行地从输入信息中选取多组信息。每个注意力关注输入信息的不同部分。

$$\text{att}((K, V), Q) = \text{att}((K, V), q_1) \oplus \dots \oplus \text{att}((K, V), q_M)$$

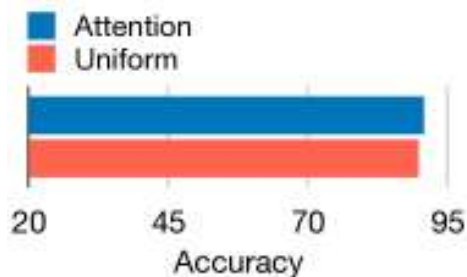
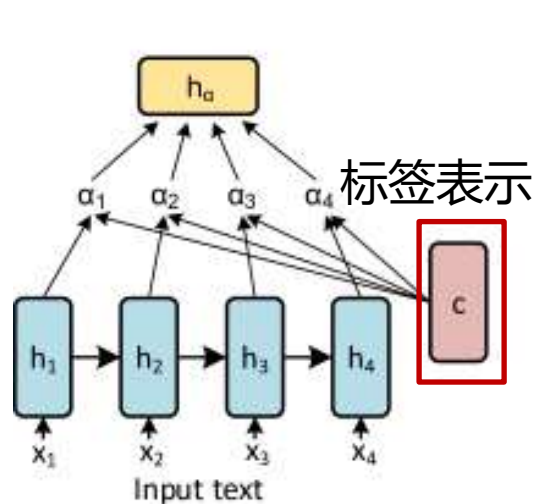
难点：怎么保证注意力的准确性



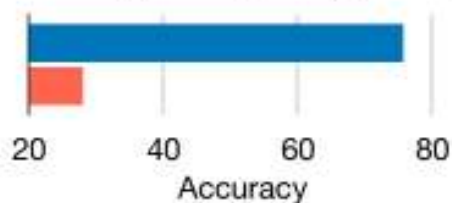
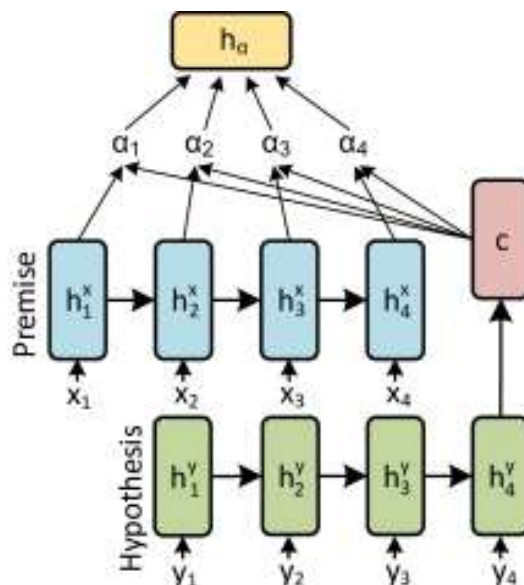
- Deep Learning 的本质：从海量数据中自动总结规律
- Attention也是Deep Learning模型需要总结规律中的一种
- 最直观的：有监督学习的额外参数



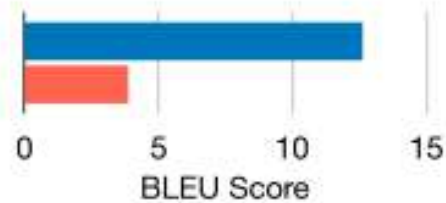
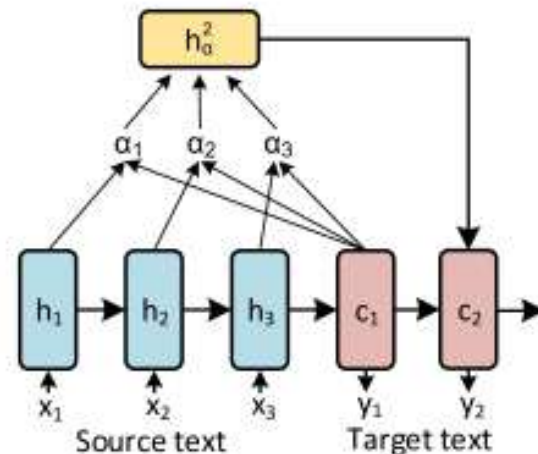
一些常见的注意力



(a) Text Classification



(b) Natural Language Inference

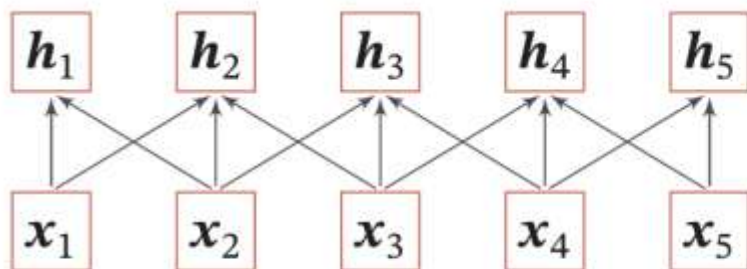


(c) Neural Machine Translation

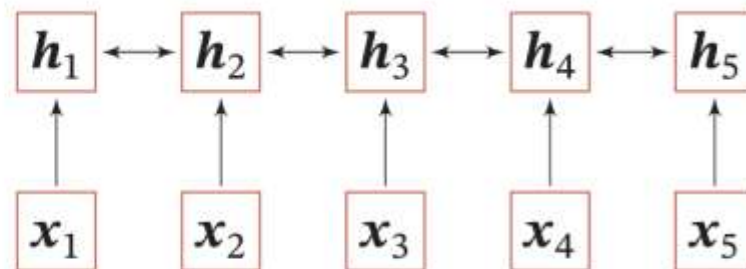
自注意力机制



- 当使用神经网络来处理一个变长的向量序列时，我们通常可以使用卷积网络或循环网络进行编码来得到一个相同长度的输出向量序列。



(a) 卷积网络



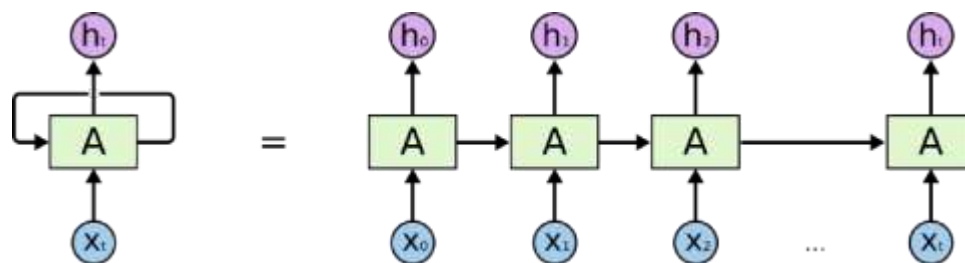
(b) 双向循环网络

只建模了输入信息的局部依赖关系

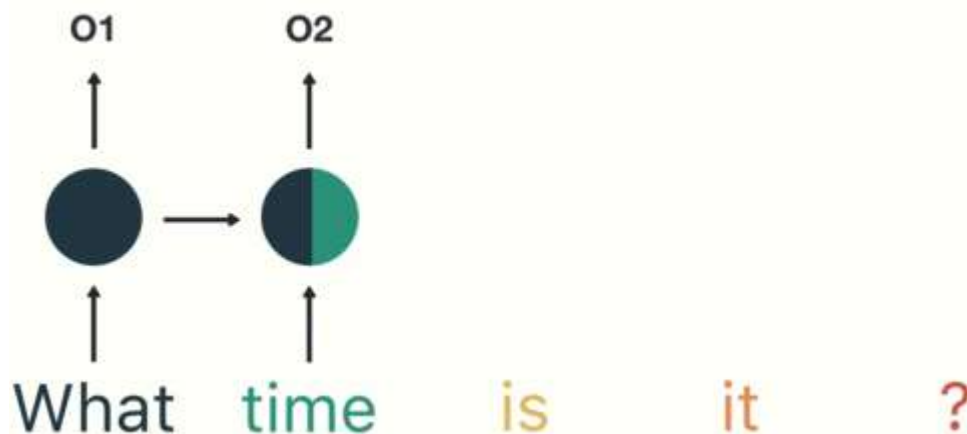
自注意力机制



- 当使用神经网络来处理一个变长的向量序列时，我们通常可以使用卷积网络或循环网络进行编码来得到一个相同长度的输出向量序列。



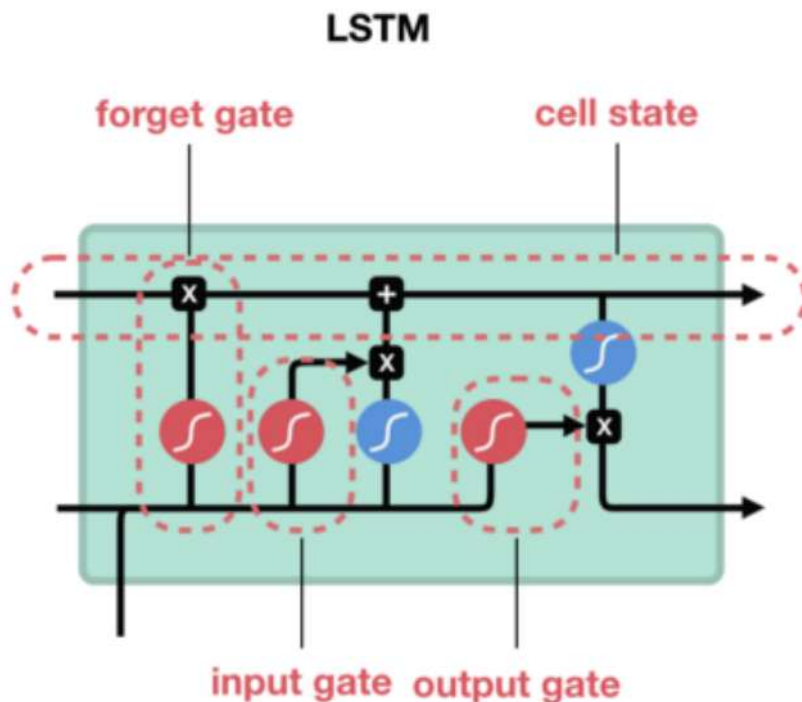
信息量变化过程



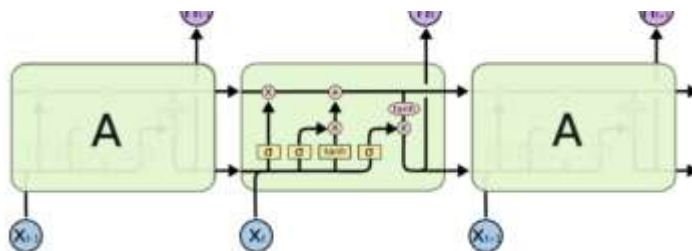
自注意力机制



- 当使用神经网络来处理一个变长的向量序列时，我们通常可以使用固定长度的输出



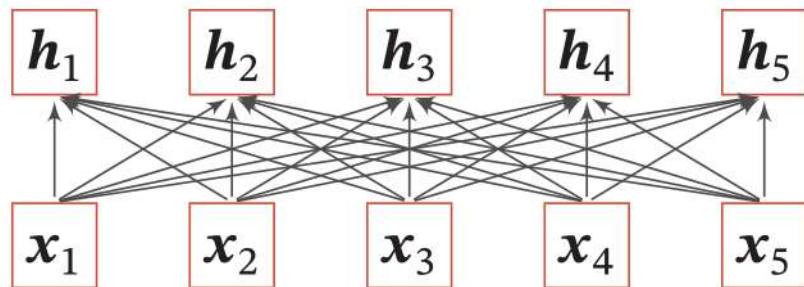
RNN注意力



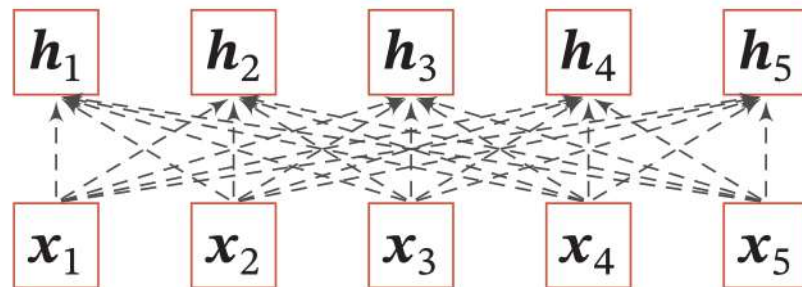
自注意力机制



- 如何建立非局部 (Non-local) 的依赖关系全连接?



(a) 全连接模型

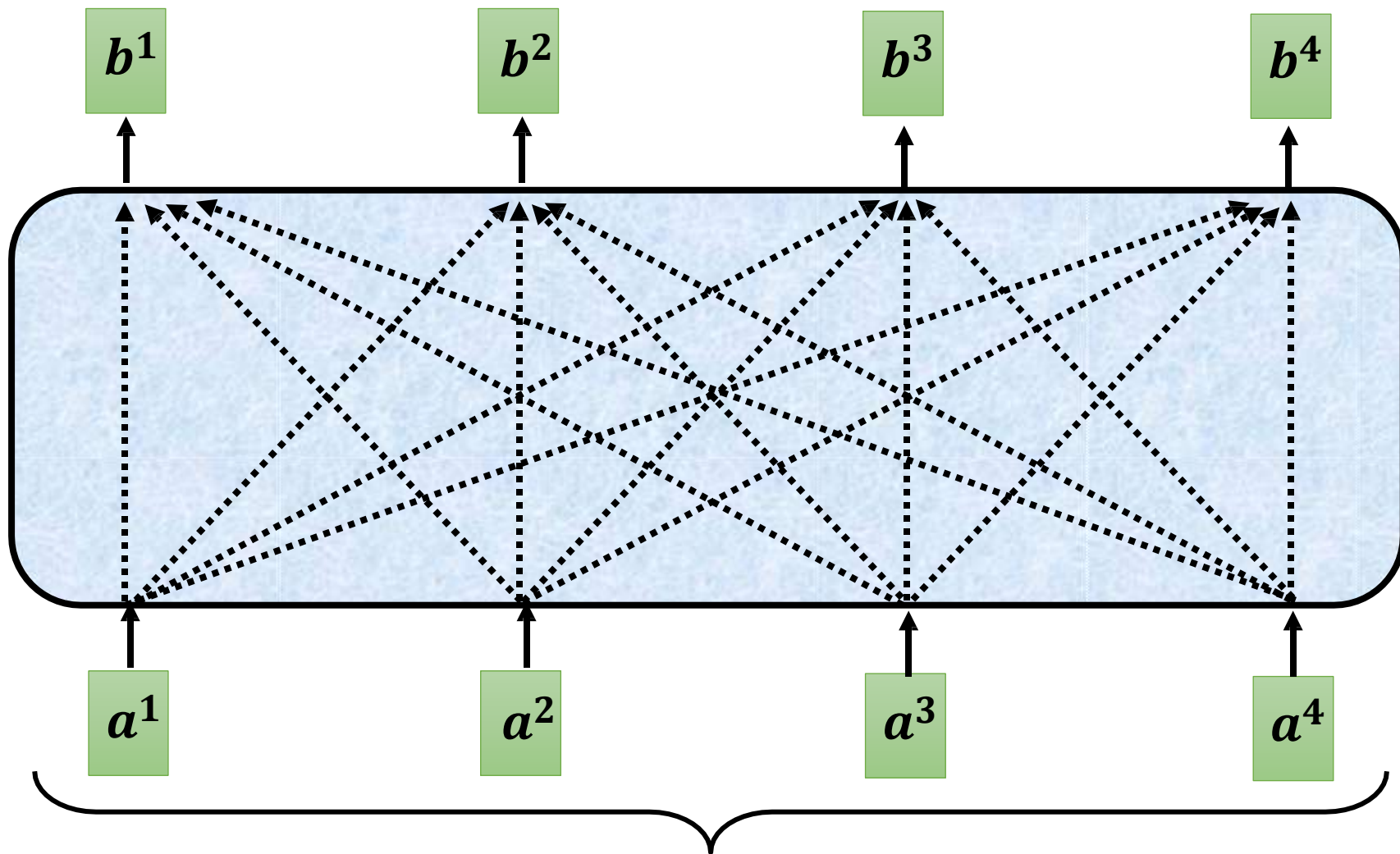


(b) 自注意力模型

无法处理变长问题，需要大量参数 连接权重 α_{ij} 由注意力机制动态生成

自注意力机制

Self-attention



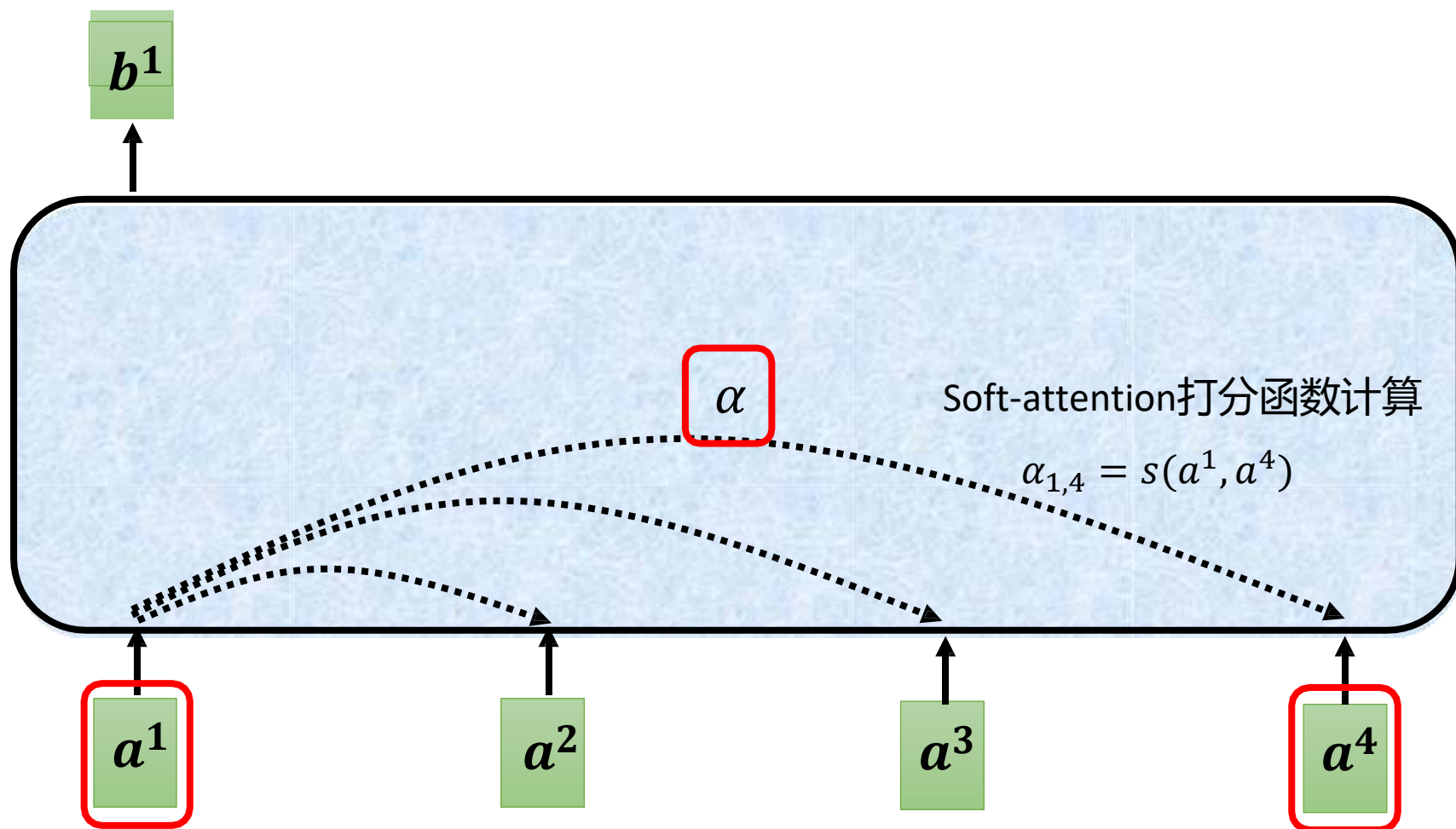
既可以是输入层，也可以是隐含层

自注意力机制



Self-attention

关键部分1：打分函数



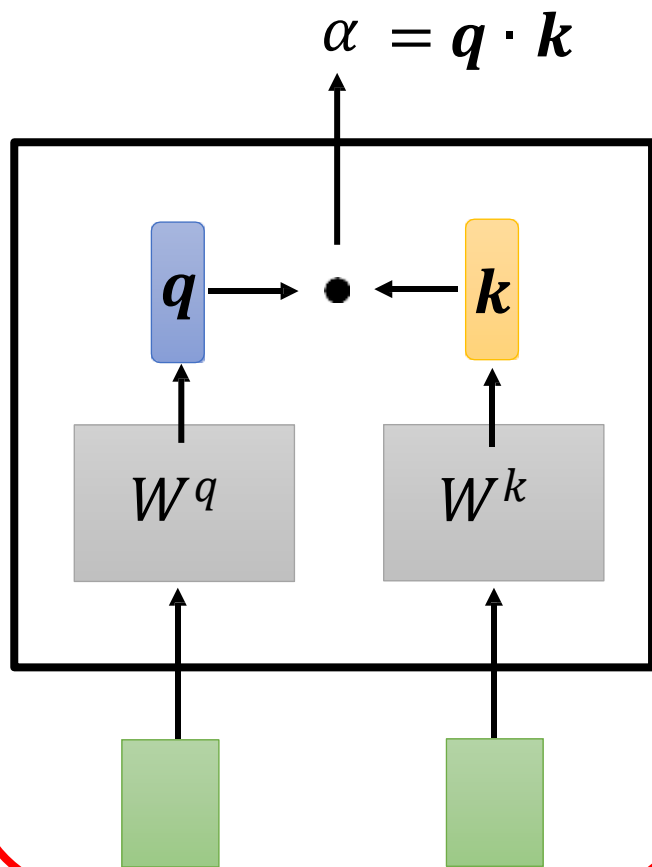
Find the relevant vectors in a sequence

自注意力机制

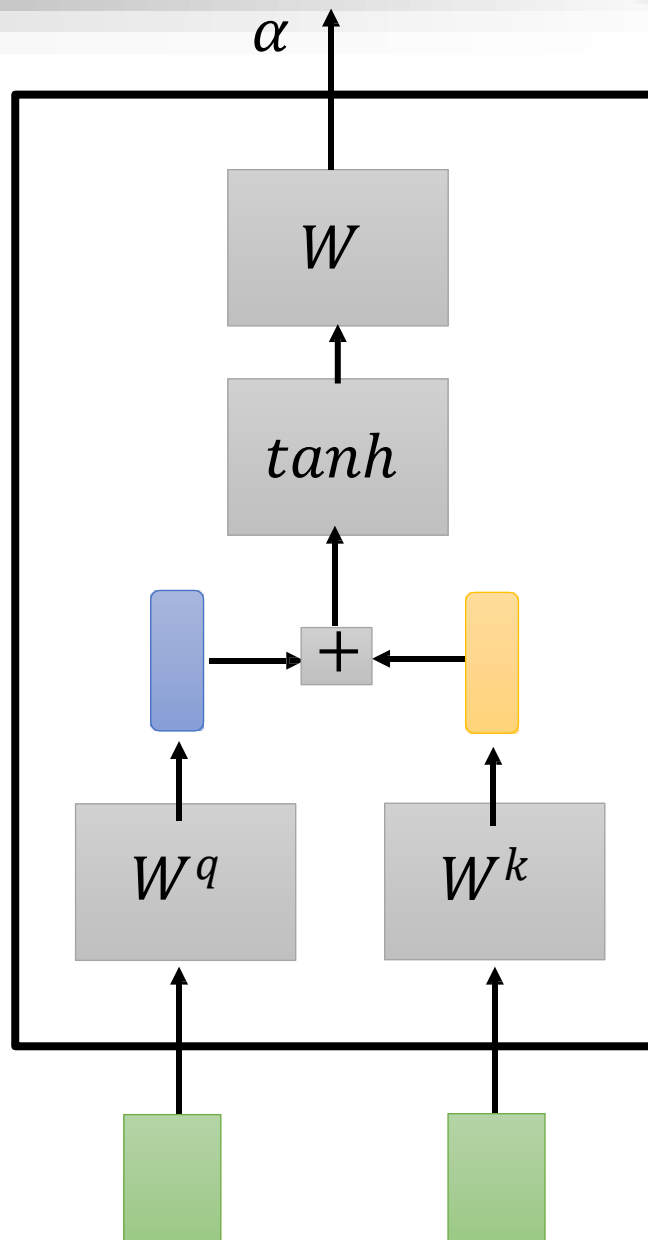


键值对打分函数计算

点乘 Dot-product



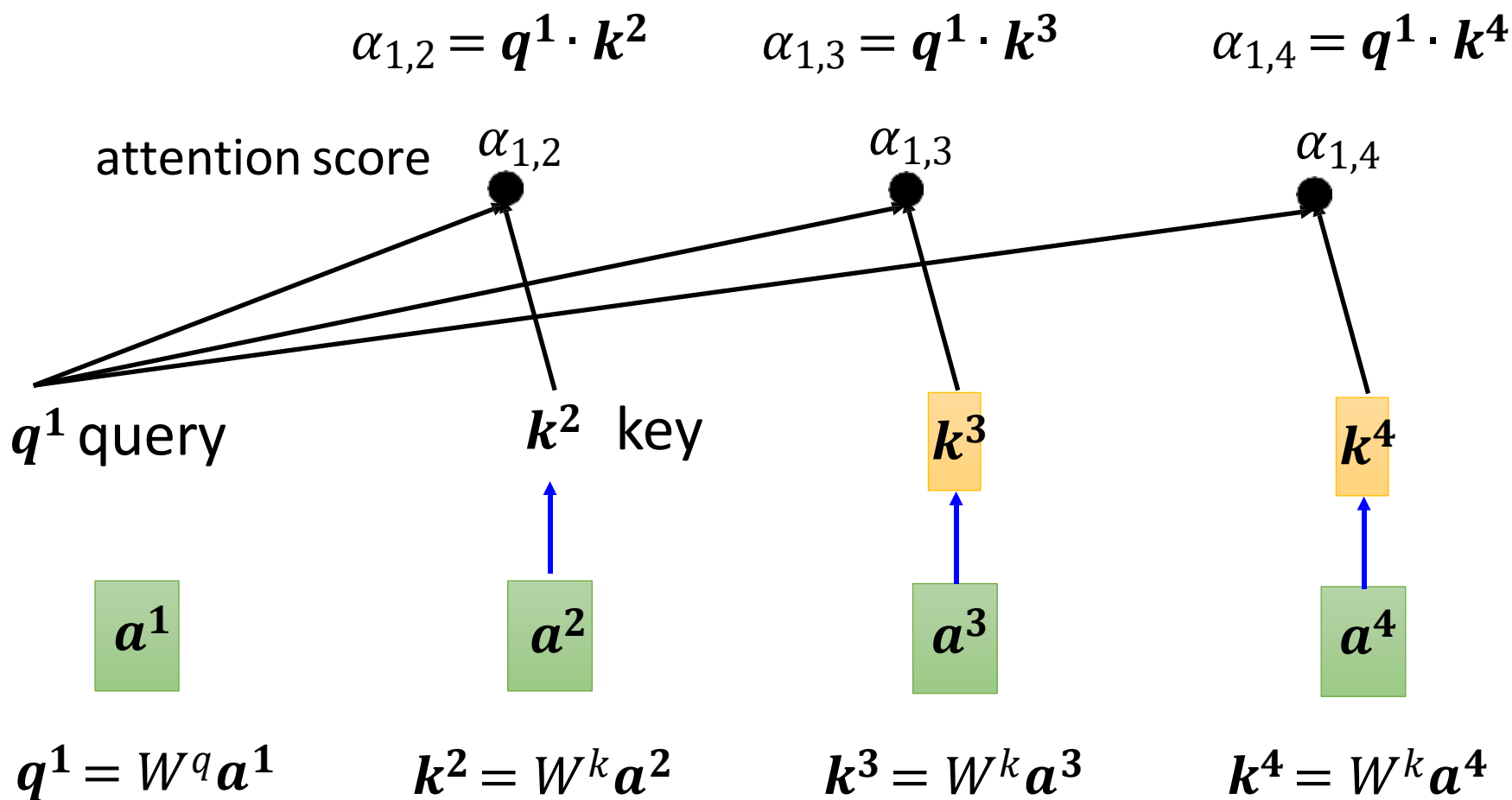
Additive



自注意力机制



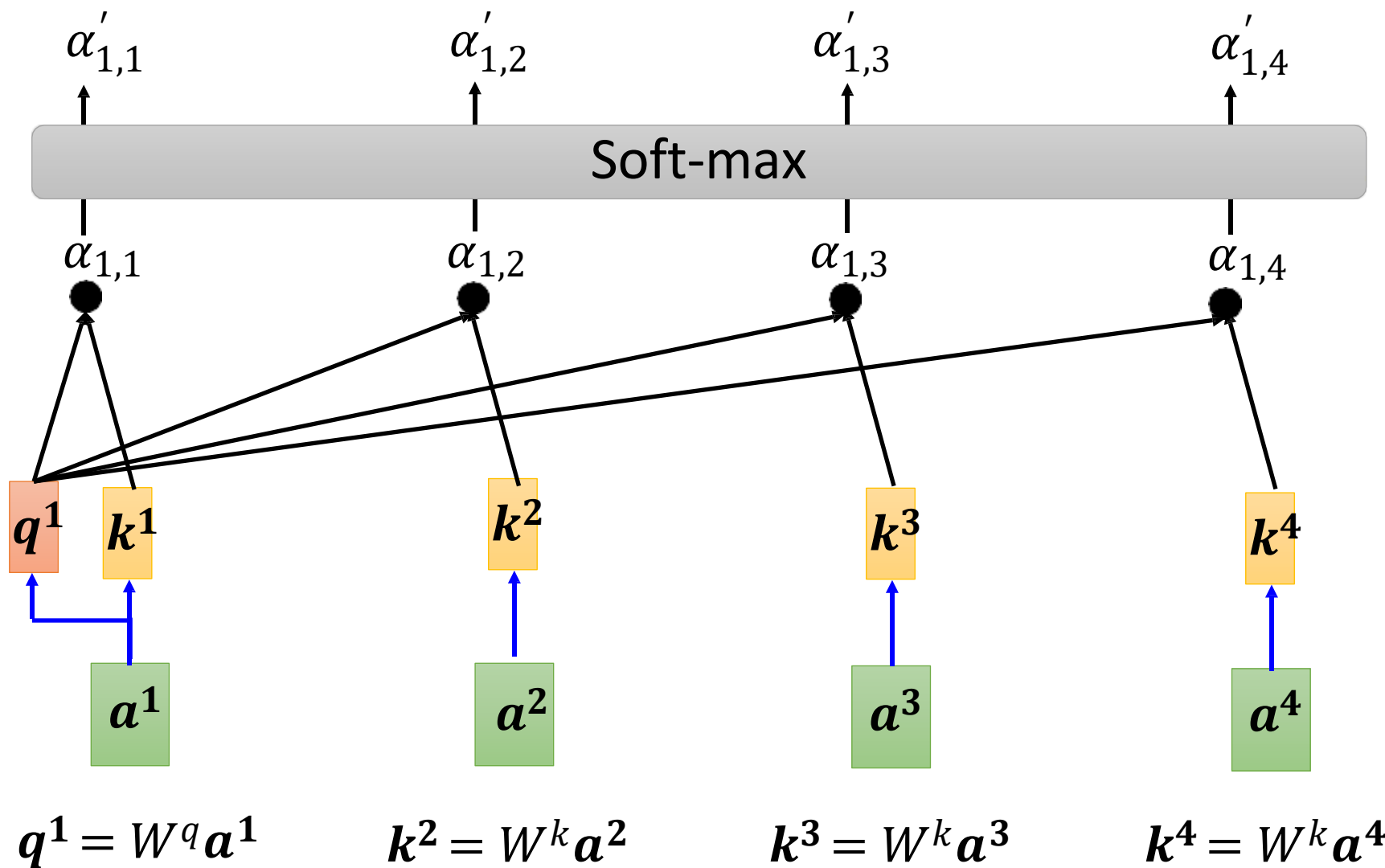
键值对打分函数计算



自注意力机制



$$\alpha'_{1,i} = \exp(\alpha_{1,i}) / \sum_j \exp(\alpha_{1,j})$$

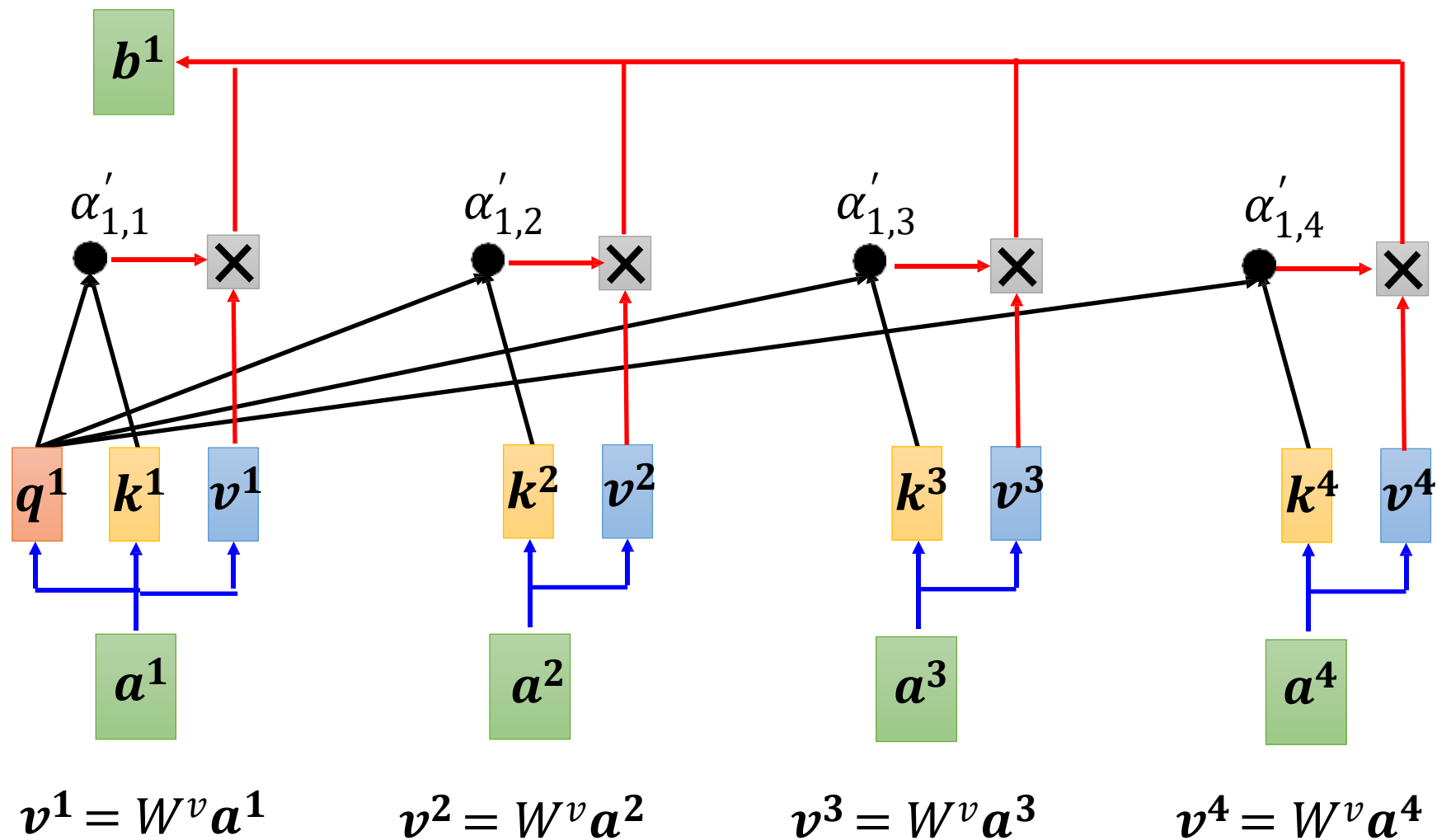


自注意力机制



基于注意力得分提取信息

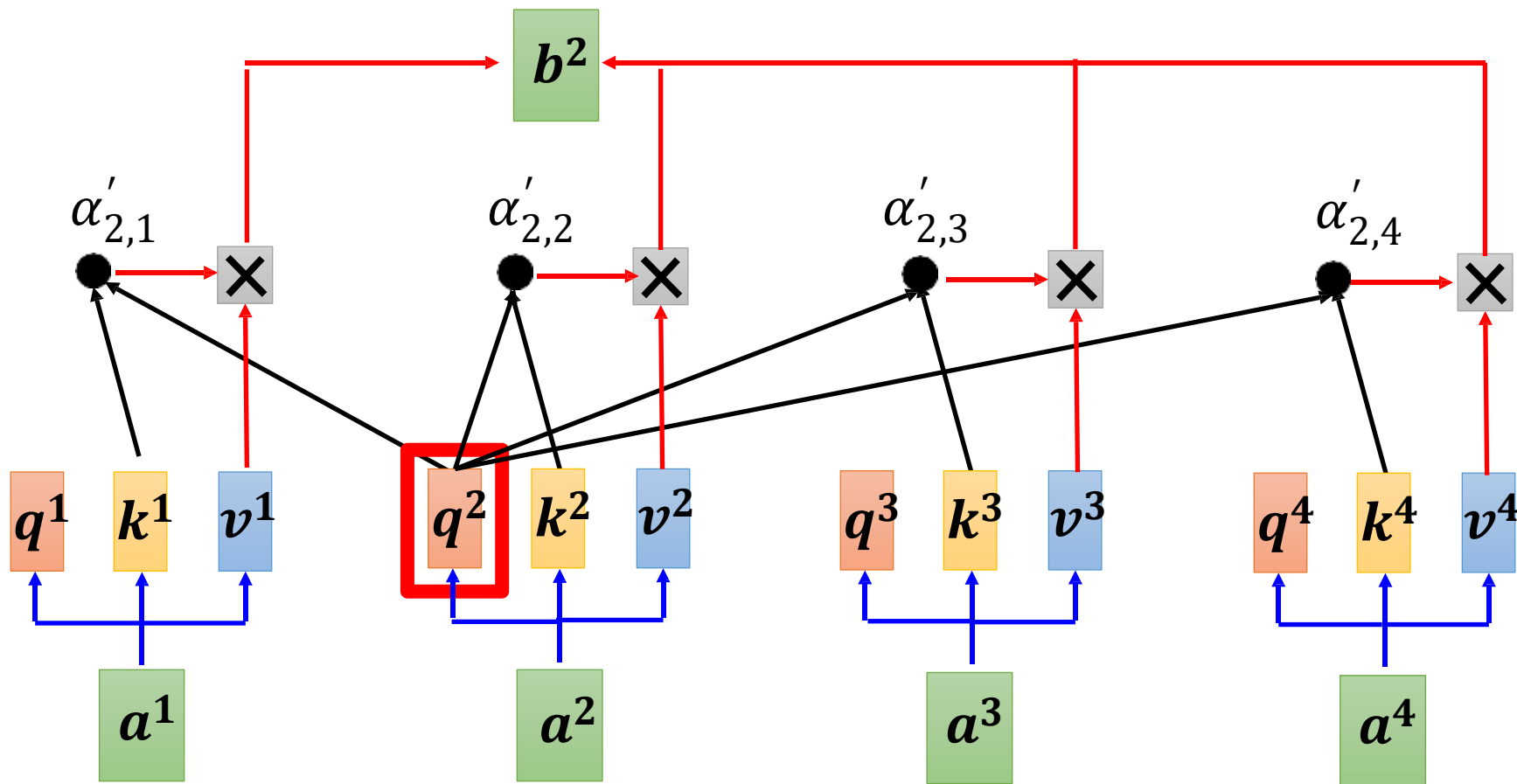
$$b^1 = \sum \alpha'_{1,i} v^i$$



自注意力机制



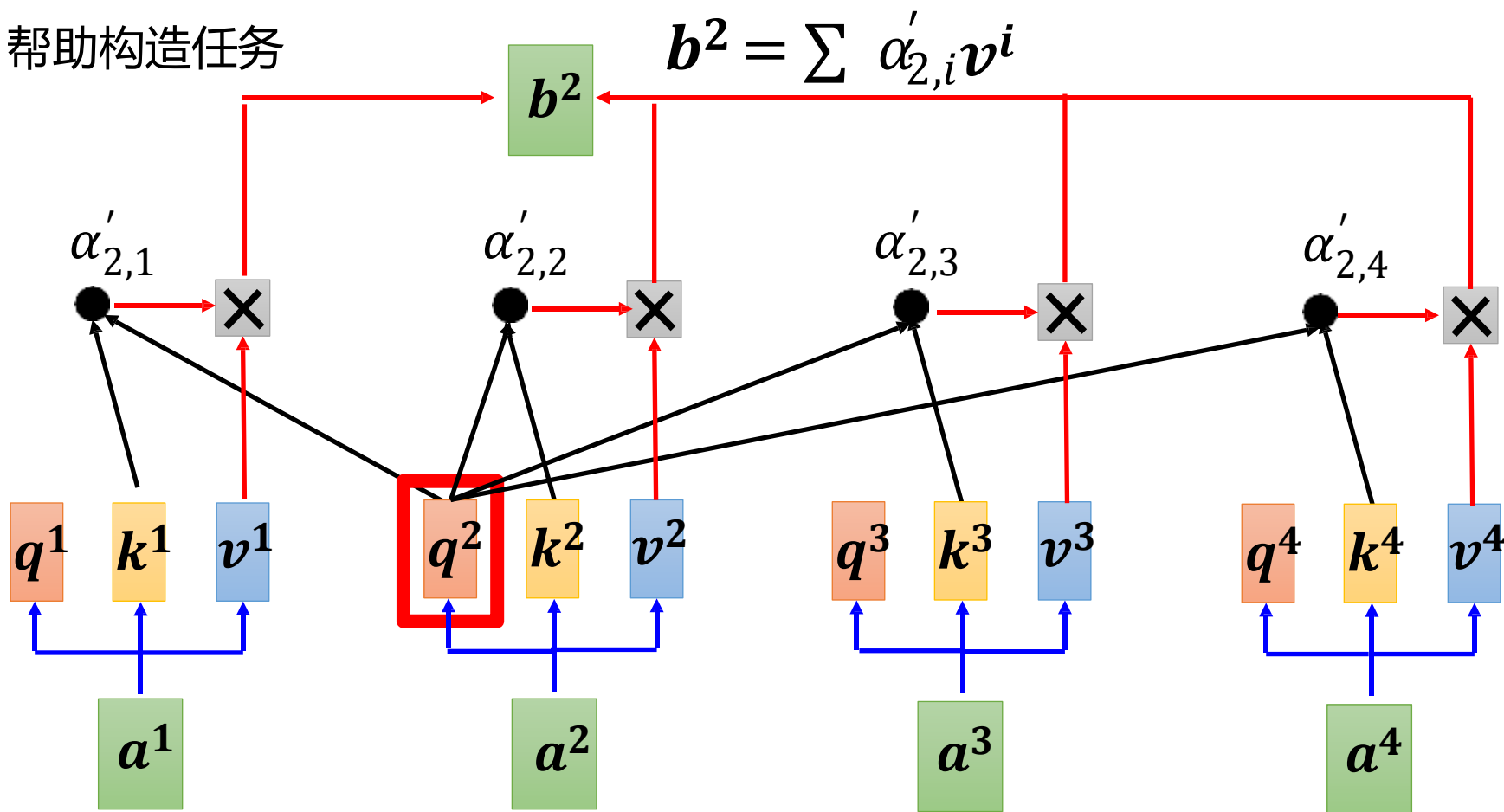
$$b^2 = \sum \alpha'_{2,i} v^i$$



自注意力小节



自注意力之所以被称为“自注意力”：任务信息来自于自身，无需外部信息来帮助构造任务



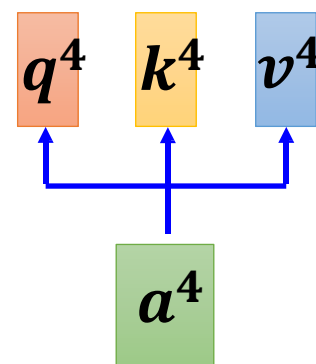
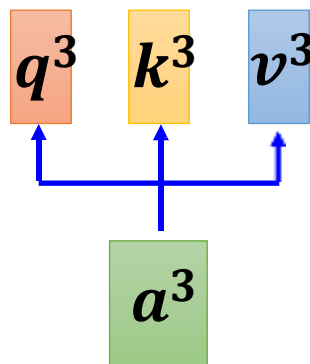
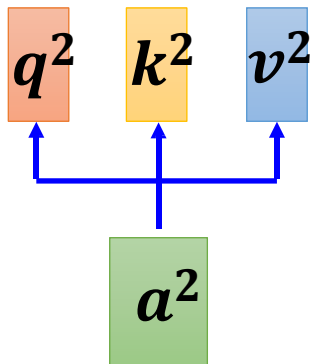
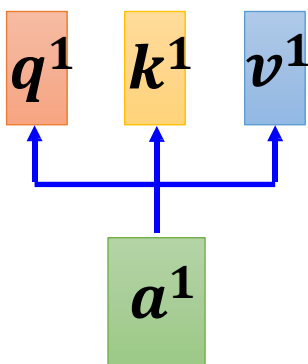
自注意力模型



$$q^i = W^q a^i \quad \begin{matrix} q^1 & q^2 & q^3 & q^4 \\ \hline Q \end{matrix} = \begin{matrix} W^q & \begin{matrix} a^1 & a^2 & a^3 & a^4 \\ \hline I \end{matrix} \end{matrix}$$

$$k^i = W^k a^i \quad \begin{matrix} k^1 & k^2 & k^3 & k^4 \\ \hline K \end{matrix} = \begin{matrix} W^k & \begin{matrix} a^1 & a^2 & a^3 & a^4 \\ \hline I \end{matrix} \end{matrix}$$

$$v^i = W^v a^i \quad \begin{matrix} v^1 & v^2 & v^3 & v^4 \\ \hline V \end{matrix} = \begin{matrix} W^v & \begin{matrix} a^1 & a^2 & a^3 & a^4 \\ \hline I \end{matrix} \end{matrix}$$

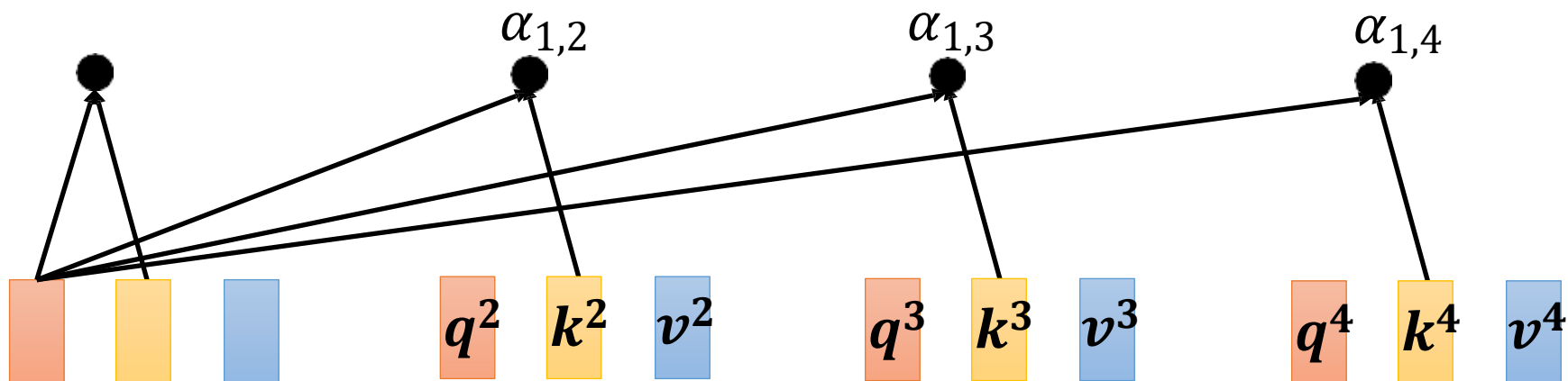


自注意力模型



$$\begin{aligned}\alpha_{1,1} &= k^1 q^1 & \alpha_{1,2} &= k^2 q^1 \\ & k^3 q^1 & \alpha_{1,4} &= k^4 q^1\end{aligned}$$

$$\begin{aligned}\alpha_{1,1} \\ \alpha_{1,2} \\ \alpha_{1,3} \\ \alpha_{1,4}\end{aligned} = \begin{aligned}k^1 \\ k^2 \\ k^3 \\ k^4\end{aligned} q^1$$



自注意力模型

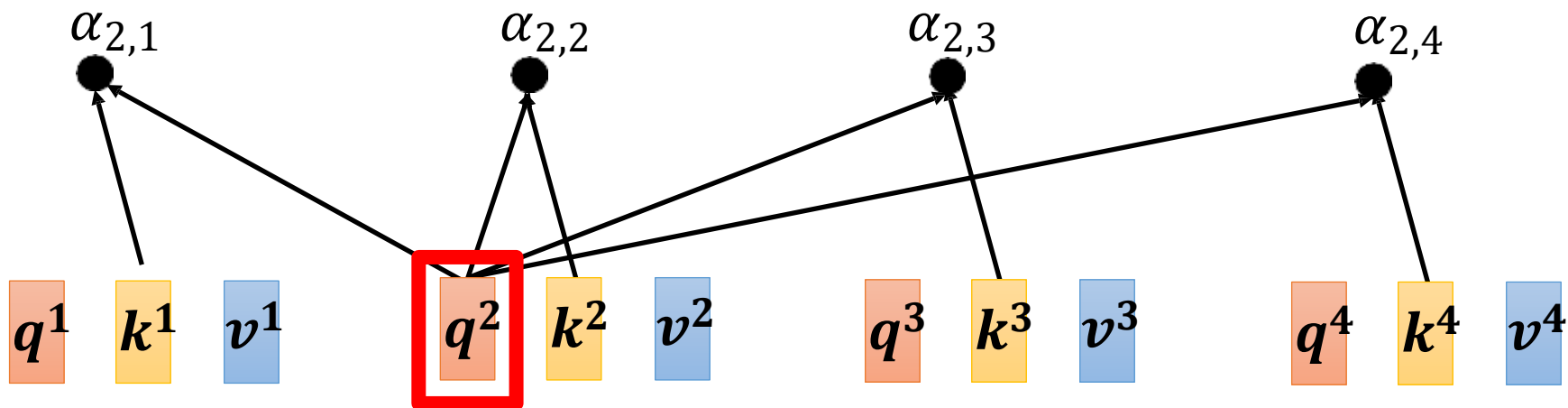


$$\alpha_{1,1} = k^1 q^1$$

$$\alpha_{1,2} = k^2 q^1$$

$$\alpha_{1,3} = k^3 q^1$$

$$\alpha_{1,4} = k^4 q^1$$

 $\alpha_{1,1}$ k^1 $\alpha_{1,2}$ k^2 $\alpha_{1,3}$ k^3 $\alpha_{1,4}$ k^4 $=$ q^1 

$\alpha'_{1,1}$	$\alpha'_{2,1}$	$\alpha'_{3,1}$	$\alpha'_{4,1}$
$\alpha'_{1,2}$	$\alpha'_{2,2}$	$\alpha'_{3,2}$	$\alpha'_{4,2}$
$\alpha'_{1,3}$	$\alpha'_{2,3}$	$\alpha'_{3,3}$	$\alpha'_{4,3}$
$\alpha'_{1,4}$	$\alpha'_{2,4}$	$\alpha'_{3,4}$	$\alpha'_{4,4}$

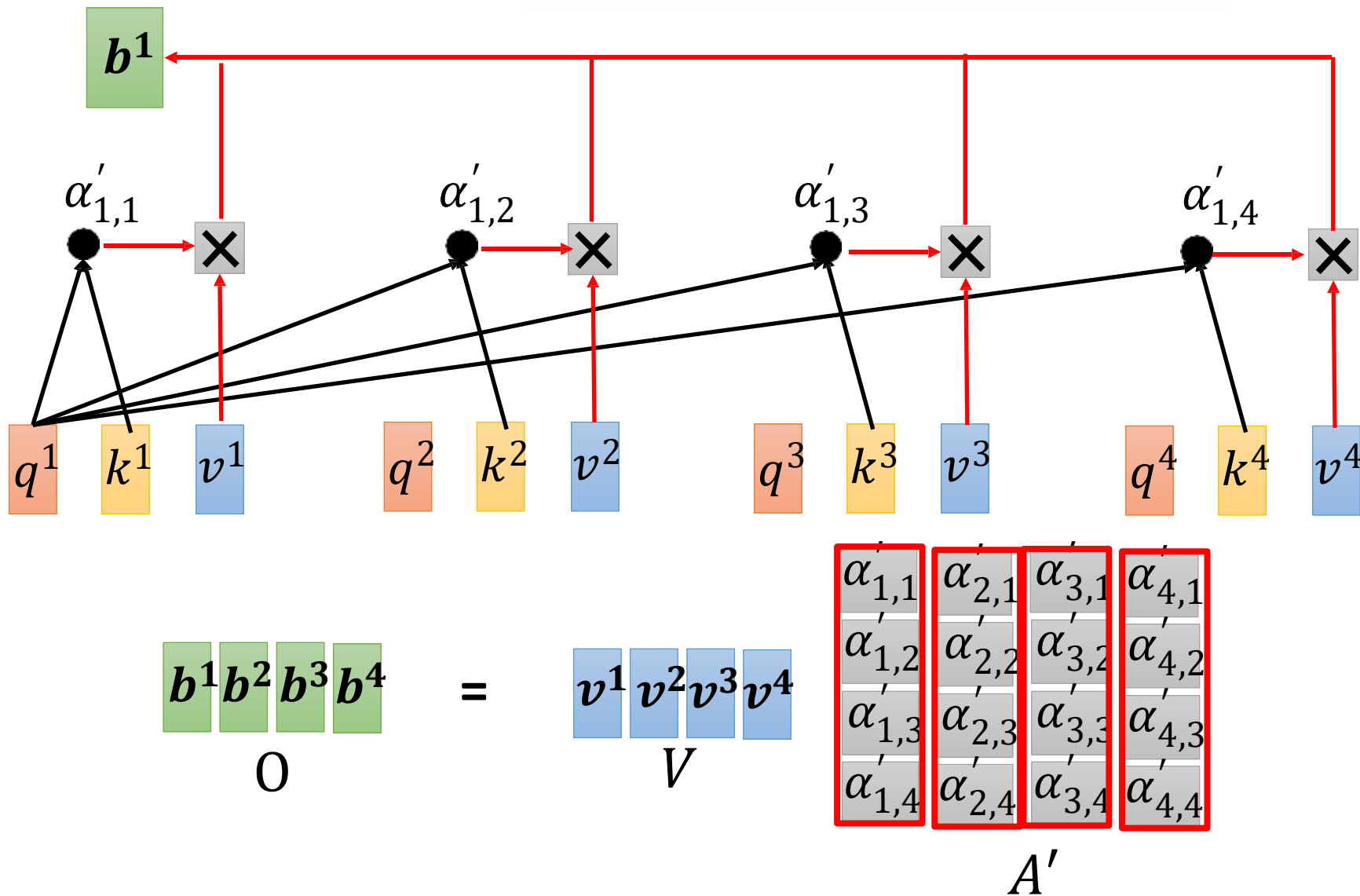


softmax

$\alpha_{1,1}$	$\alpha_{2,1}$	$\alpha_{3,1}$	$\alpha_{4,1}$
$\alpha_{1,2}$	$\alpha_{2,2}$	$\alpha_{3,2}$	$\alpha_{4,2}$
$\alpha_{1,3}$	$\alpha_{2,3}$	$\alpha_{3,3}$	$\alpha_{4,3}$
$\alpha_{1,4}$	$\alpha_{2,4}$	$\alpha_{3,4}$	$\alpha_{4,4}$

 $=$ k^1 k^2 k^3 k^4 K^T q^1 q^2 q^3 q^4 Q

自注意力模型



自注意力模型

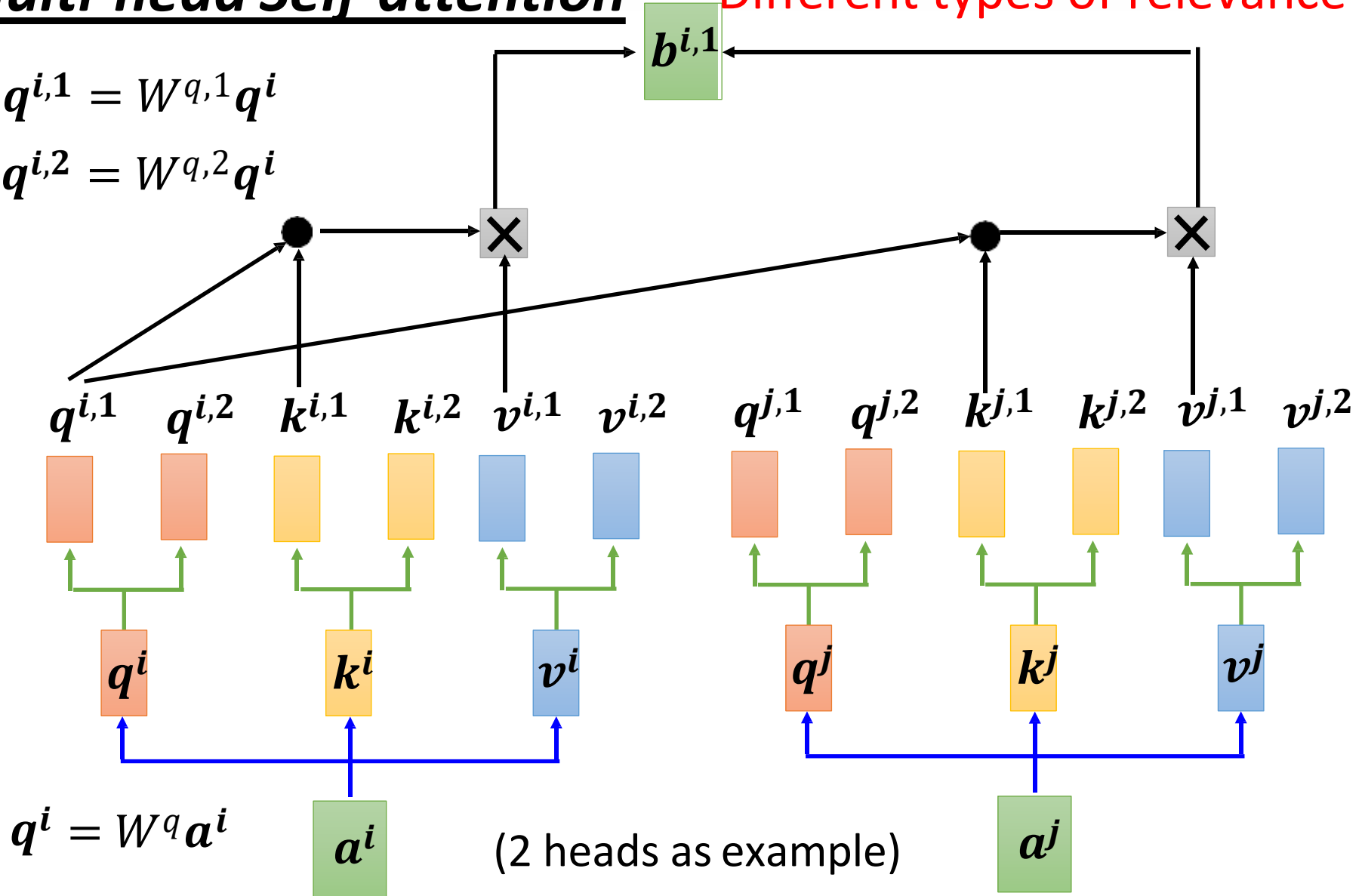


Multi-head Self-attention

Different types of relevance

$$q^{i,1} = W^{q,1} q^i$$

$$q^{i,2} = W^{q,2} q^i$$



自注意力模型



Multi-head Self-attention

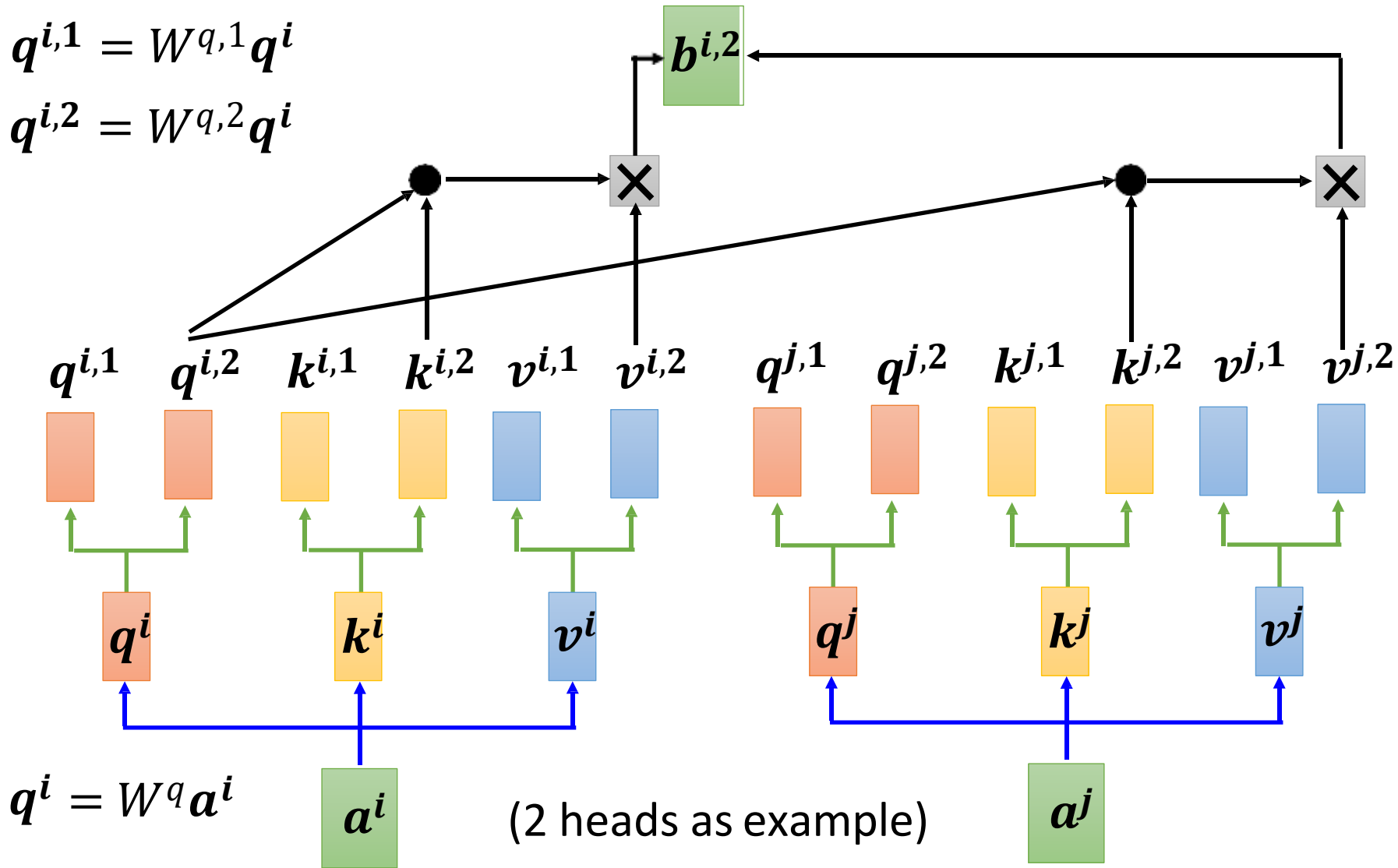
$b^{i,1}$

$b^{i,2}$

Different types of relevance

$$q^{i,1} = W^{q,1} q^i$$

$$q^{i,2} = W^{q,2} q^i$$



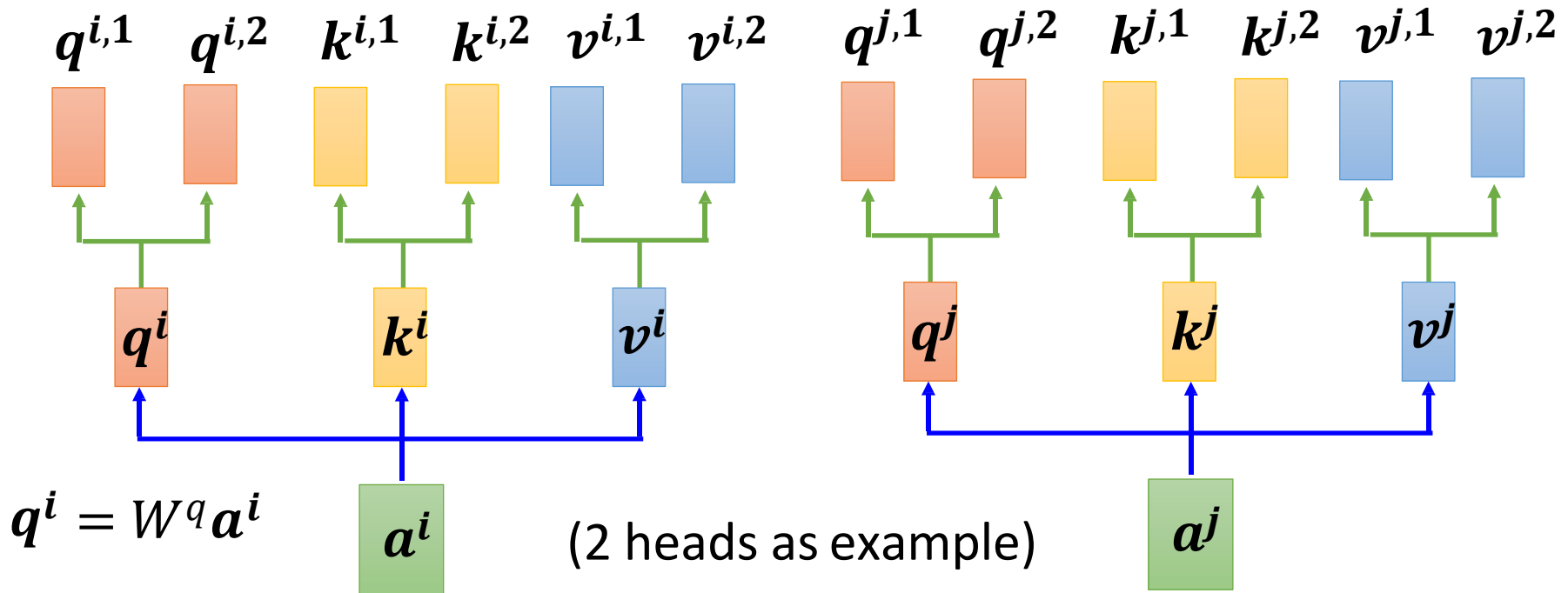
自注意力模型



Multi-head Self-attention

Different types of relevance

$$b^i = W^O \begin{bmatrix} b^{i,1} \\ b^{i,2} \end{bmatrix}$$



自注

序号	比较维度	单头自注意力	多头自注意力	简要说明
1	注意力头的数量	1	多个 (如8、16等)	多头机制能够同时关注输入序列的不同部分。
2	信息捕捉能力 ⁺	有限	丰富	多头机制通过不同的头捕捉不同的特征，信息更丰富。
3	表达能力	相对较弱	相对较强	多头机制提高了模型的表达能力，有助于处理复杂任务。
4	泛化性能	一般	较好	多头机制增强了模型的泛化能力，对新数据的适应性更强。
5	计算复杂度 ⁺	较低	较高	多头机制增加了计算量，但通常通过并行计算来优化。
6	训练难度 ⁺	相对简单	相对复杂	多头机制需要更多的训练数据和计算资源来优化。
7	应用场景	简单任务	复杂任务	多头机制在处理复杂任务时表现更优，如机器翻译、文本生成等。
8	模型大小	较小	较大	多头机制通常需要更大的模型容量来存储更多的参数。





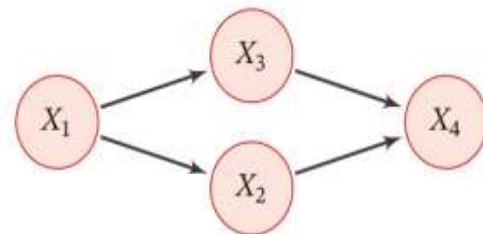
概率图模型



贝叶斯网络



有向图模型 (Directed Graphical model), 也称为贝叶斯网络 (Bayesian Network), 或信念网络



定义 11.1

有向非循环图 G , G 中的每个节点都对应一个随机变量, 每个连接 e_{ij} 表示

$$p(x) = p(x | z, y)p(z, y) = p(x | z)p(z | y)p(y)$$

所有父节点变量取值, $p(x_k | \mathbf{x}_{\pi_k})$ 表示每个随机变量 X_k 的局部条件概率分布

(Local Conditional Probability Distribution). 如果 $\mathbf{X}$ 的联合概率分布可以分解为每个随机变量 X_k 的局部条件概率的连乘形式, 即

$$p(\mathbf{x}) = \prod_{k=1}^K p(x_k | \mathbf{x}_{\pi_k}), \quad (11.8)$$

那么 $(G, \mathbf{X})$ 构成了一个贝叶斯网络.

局部马尔可夫性质



- 贝叶斯网络的局部马尔可夫性质：每个随机变量在给定父节点的情况下，条件独立于它的非后代节点。

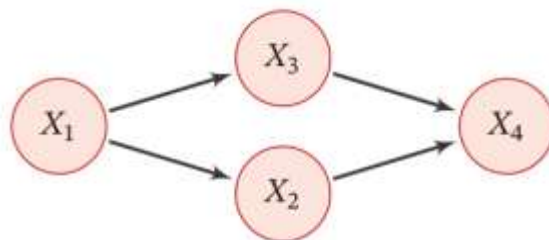


和
从而降

简化，

$$\begin{aligned} p(x) &= p(x | z, y) p(z, y) = p(x | z) p(z | y) p(y) \\ &= p(x_1) p(x_2 | x_1) p(x_3 | x_1) p(x_4 | x_2, x_3), \end{aligned}$$

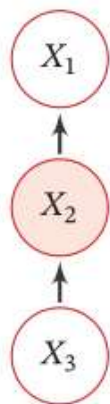
$P(x)$ 是4个局部条件概率的乘积，这样只需要 $1 + 2 + 2 + 4 = 9$ 个独立参数。



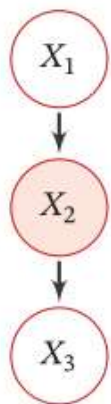
条件独立性



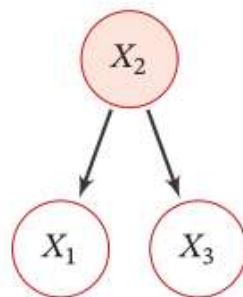
- 在贝叶斯网络中，如果两个节点是直接连接的，它们肯定是非条件独立的，是直接因果关系。
- 父节点是“因”，子节点是“果”。
- 如果两个节点不是直接连接的，但是它们之间有一条经过其他节点的路径连接互连接，它们之间的条件独立性就比较复杂。



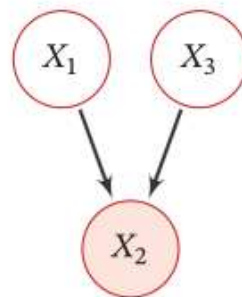
(a)



(b)

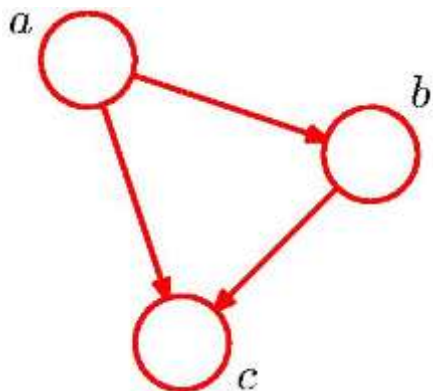


(c)

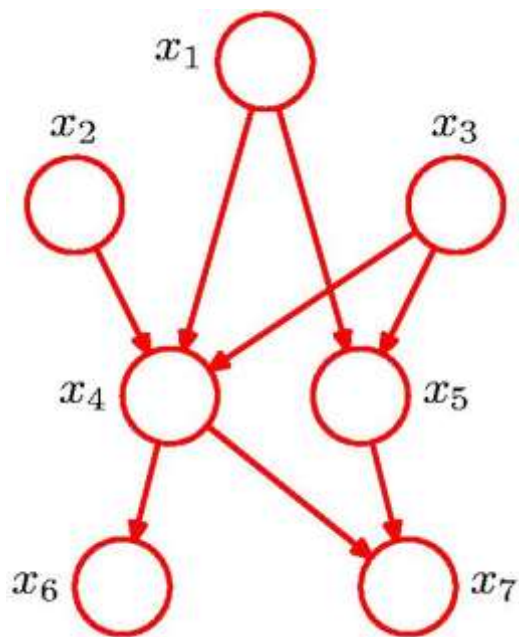


(d)

练习



$$p(a, b, c) = p(c|a, b)p(a, b) = p(c|a, b)p(b|a)p(a)$$



$$p(x_1, \dots, x_7) = p(x_1)p(x_2)p(x_3)p(x_4|x_1, x_2, x_3) \\ p(x_5|x_1, x_3)p(x_6|x_4)p(x_7|x_4, x_5)$$



基于概率图的参数学习与 模型推断



参数学习



导入例子：朴素贝叶斯

Text	Category
A great game (一个伟大的比赛)	Sports (体育运动)
The election was over. (选举结束)	Not sports (不是体育运动)
Very clean match (没内幕的比赛)	Sports (体育运动)
A clean but forgettable game (一场难以忘记的比赛)	Sports (体育运动)
It was a close election (这是一场势均力敌的选举)	Not sports (不是体育运动)

$$p(y | x_1, x_2, x_3) = \frac{p(x_1, x_2, x_3 | y)p(y)}{p(x_1, x_2, x_3)} \propto p(x_1, x_2, x_3 | y)$$

$$p(x_1, x_2, x_3 | y) = p(x_1 | y)p(x_2 | y)p(x_3 | y)$$

在简单数据集中可以直接count!

Word	P(word Sports)	P(word Not Sports)
a	$\frac{2+1}{11+14}$	$\frac{1+1}{9+14}$
very	$\frac{1+1}{11+14}$	$\frac{0+1}{9+14}$
close	$\frac{0+1}{11+14}$	$\frac{1+1}{9+14}$
game	$\frac{2+1}{11+14}$	$\frac{0+1}{9+14}$

能否通过参数估计的形式把上面这张表格估计出来呢?
我们在统计的时候实际就是在做最大似然

参数学习



导入例子：朴素贝叶斯

Text	Category
A great game (一个伟大的比赛)	Sports (体育运动)
The election was over. (选举结束)	Not sports (不是体育运动)
Very clean match (没内幕的比赛)	Sports (体育运动)
A clean but forgettable game (一场难以忘记的比赛)	Sports (体育运动)
It was a close election (这是一场势均力敌的选举)	Not sports (不是体育运动)

$$p(y | x_1, x_2, x_3) = \frac{p(x_1, x_2, x_3 | y)p(y)}{p(x_1, x_2, x_3)} \propto p(x_1, x_2, x_3 | y)$$

$$p(x_1, x_2, x_3 | y) = p(x_1 | y)p(x_2 | y)p(x_3 | y)$$

我们先做如下的参数定义：

定义 $\tilde{X} = (x_1, x_2, \dots, x_N)$, N 表示训练样本的数量

其中, $x_i = (x_i^{(1)}, \dots, x_i^{(n)})^T$

Y 的取值范围为 $C = \{c_1, c_2\}$, $c_k \in \{0, 1\}$

其中变量 $X^{(j)}$ 的取值范围为 $\{w_1, \dots, w_V\}$

定义参数 $\theta_\ell^{c_k}$ 表示词 $p(X^{(j)} = w_\ell | y = c_k)$ 这个概率

怎么用这个定义来表示整个数据集的概率表示呢？



参数学习



定义 $\tilde{X} = (x_1, x_2, \dots, x_N)$, N 表示训练样本的数量

其中, $x_i = (x_i^{(1)}, \dots, x_i^{(n)})^T$

Y 的取值范围为 $C = \{c_1, c_2\}$, $c_k \in \{0, 1\}$

其中变量 $X^{(j)}$ 的取值范围为 $\{w_1, \dots, w_V\}$

定义参数 $\theta_\ell^{c_k}$ 表示词 $p(X^{(j)} = w_\ell \mid y = c_k)$

$$p(\tilde{X} \mid Y) = p(\tilde{X} \mid \Theta) = \prod_{i=1}^N p(x_i \mid \Theta) \propto \sum_{i=1}^N \log p(x_i \mid \Theta)$$

$$\mathcal{L} = \sum_{i=1}^N \log p(x_i \mid \Theta) = \sum_{i=1}^N \sum_{j=1}^n \sum_{k=1}^K \sum_{\ell=1}^V s_{j,\ell}^{c_k} \log \theta_\ell^{c_k}$$

其中 $s_{j,\ell}^{c_k} \in \{0, 1\}$, 表示句子 i 是否属于当前的类别 c_k 且第 j 个词为 w_ℓ ,

也可以表示成 $s_{j,\ell}^{c_k} = I(x_i^{(j)} = w_\ell, y_i = c_k)$ **求解参数 $\theta_\ell^{c_k}$**

$$\mathcal{L}' = \sum_{j=1}^n \sum_{k=1}^K \sum_{\ell=1}^V n_{j,\ell}^{c_k} \log \theta_\ell^{c_k} - \lambda \left(\sum_{\ell=1}^V \theta_\ell^{c_k} - 1 \right) \longrightarrow \frac{\partial \mathcal{L}'}{\partial \theta_\ell^{c_k}} = \frac{\partial \mathcal{L}'}{\partial \lambda} = 0$$

其中 $n_{j,\ell}^{c_k} = \sum_{i=1}^N I(x_i^{(j)} = w_\ell, y_i = c_k)$

参数学习



定义 $\tilde{X} = (x_1, x_2, \dots, x_N)$, N 表示训练样本的数量

其中, $x_i = (x_i^{(1)}, \dots, x_i^{(n)})^T$

Y 的取值范围为 $C = \{c_1, c_2\}$, $c_k \in \{0, 1\}$

定义参数 $\theta_\ell^{c_k}$ 表示 $p(x_i^{(n)} = a_\ell \mid y = c_k)$ 这个概率, ℓ 的取值范围为 $[1, V]$

求解参数 $\theta_\ell^{c_k}$

$$\mathcal{L}' = \sum_{j=1}^n \sum_{k=1}^K \sum_{\ell=1}^V n_{j,\ell}^{c_k} \log \theta_\ell^{c_k} - \lambda \left(\sum_{\ell=1}^V \theta_\ell^{c_k} - 1 \right) \longrightarrow \frac{\partial \mathcal{L}'}{\partial \theta_\ell^{c_k}} = \frac{\partial \mathcal{L}'}{\partial \lambda} = 0$$

其中 $n_{j,\ell}^{c_k} = \sum_{i=1}^N I(x_i^{(j)} = w_\ell, y_i = c_k)$

$$\begin{cases} \sum_{j=1}^n \frac{n_{j,1}^{c_k=0}}{\theta_1^0} - \lambda = 0 \\ \sum_{j=1}^n \frac{n_{j,2}^{c_k=0}}{\theta_2^0} - \lambda = 0 \\ \dots \end{cases}$$

代入

$$\sum_{\ell} \theta_\ell^{c_k} - 1 = 0$$

.....

$$\theta_1^0 = \frac{\sum_{j=1}^n n_{j,1}^{c_k=0}}{\sum_{\ell=1}^V \sum_{j=1}^n n_{j,1}^{c_k=0}} = \frac{\sum_{j=1}^n \sum_{i=1}^N I(x_i^j = w_1, y_i = 0)}{\sum_{\ell=1}^V \sum_{j=1}^n \sum_{i=1}^N I(x_i^j = w_\ell, y_i = 0)}$$

参数学习



导入例子：朴素贝叶斯

Text	Category
A great game (一个伟大的比赛)	Sports (体育运动)
The election was over. (选举结束)	Not sports (不是体育运动)
Very clean match (没内幕的比赛)	Sports (体育运动)
A clean but forgettable game (一场难以忘记的比赛)	Sports (体育运动)
It was a close election (这是一场势均力敌的选举)	Not sports (不是体育运动)

$$p(y | x_1, x_2, x_3) = \frac{p(x_1, x_2, x_3 | y)p(y)}{p(x_1, x_2, x_3)} \propto p(x_1, x_2, x_3 | y)$$

$$p(x_1, x_2, x_3 | y) = p(x_1 | y)p(x_2 | y)p(x_3 | y)$$

在简单数据集中可以直接count!

Word	P(word Sports)	P(word Not Sports)
a	$\frac{2+1}{11+14}$	$\frac{1+1}{9+14}$
very	$\frac{1+1}{11+14}$	$\frac{0+1}{9+14}$
close	$\frac{0+1}{11+14}$	$\frac{1+1}{9+14}$
game	$\frac{2+1}{11+14}$	$\frac{0+1}{9+14}$

分布参数化

$$p(x | y) \rightarrow p_{\theta}(x | y), p(x | y; \theta)$$

$$p(x_1, x_2, x_3 | y) = p(x_1 | y)p(x_2 | y)p(x_3 | y) = \prod p_{\theta}(x | y) \propto \sum \log p_{\theta}(x | y)$$

似然函数

结果推断



给定一个新的样本如何做标签的预测? $(x_1^{test}, x_2^{test}, x_3^{test})$

$$p(y | x_1, x_2, x_3) = \frac{p(x_1, x_2, x_3 | y)p(y)}{p(x_1, x_2, x_3)} \propto p(x_1^{test}, x_2^{test}, x_3^{test} | y)$$

简单查表

基于学习的参数进行目标变量的推断

Word	P(word Sports)	P(word Not Sports)
a	$\frac{2+1}{11+14}$	$\frac{1+1}{9+14}$
very	$\frac{1+1}{11+14}$	$\frac{0+1}{9+14}$
close	$\frac{0+1}{11+14}$	$\frac{1+1}{9+14}$
game	$\frac{2+1}{11+14}$	$\frac{0+1}{9+14}$

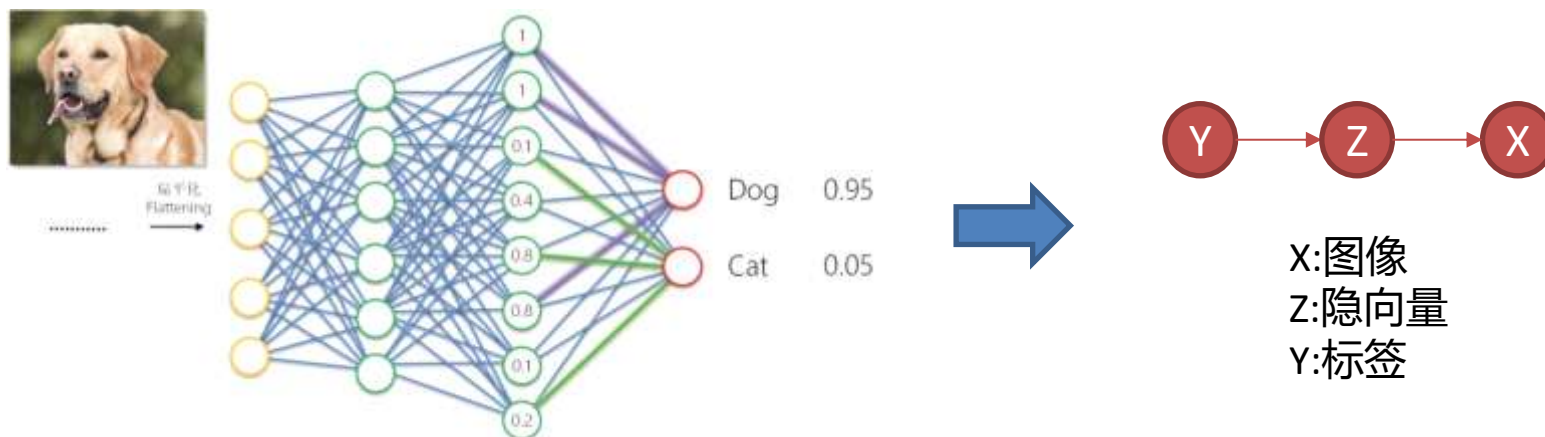
$$p(X^1 = x_1 | y = 1) = \frac{1}{\sqrt{2\pi\sigma_k^2}} \exp\left(-\frac{(x_1 - \mu_1)^2}{2\sigma_1^2}\right)$$



基于隐变量的学习和推断



隐变量定义



- 隐变量会如何影响学习和推断过程？
- 隐变量对于可观测变量 x 分布学习的影响：

$$p(x) = \int p(x | y)p(y) dy = \int \int p(x | z)p(z | y)p(y)dz dy$$

- 隐变量对于目标变量 y 分布推断的影响：

$$p(y | x) = \frac{p(x | y)p(y)}{p(x)} = \frac{\int \int p(x | z)p(z | y)p(y)dz}{p(x)}$$

主要都是
积分和条
件分布的
问题？

含隐变量的参数学习



- 隐变量即变量是不可观测的



假设参数 z 的分布的参数是 θ

- 点估计：最大似然和最大后验法

$$\hat{\Theta}_{MLE} = \arg \max_{\Theta} p(x | \Theta) \quad \hat{\Theta}_{MAP} = \arg \max_{\Theta} p(\Theta | X, \alpha)$$

需要用EM算法进行参数估计

- 贝叶斯估计：最大化概率期望

$$\begin{aligned} \hat{\Theta}_B &= E_{\Theta \sim p(\Theta | X, \alpha)}[\Theta] \\ &= \int \Theta p(\Theta | X, \alpha) d\Theta \\ &= \int \Theta \left[\frac{p(X|\Theta)p(\Theta|\alpha)}{\int p(X|\Theta)p(\Theta|\alpha)d\Theta} \right] d\Theta \end{aligned}$$

含隐变量的参数学习



- 点估计：最大似然和最大后验法

$$\hat{\Theta}_{MLE} = \arg \max_{\Theta} p(x | \Theta)$$

$$p(\tilde{X} | Y) = p(\tilde{X} | \Theta) = \prod_{i=1}^N p(x_i | \Theta) \propto \sum_{i=1}^N \log p(x_i | \Theta)$$

$$\hat{\Theta}_{MAP} = \arg \max_{\Theta} p(\Theta | X, \alpha)$$

$$P(\Theta | \tilde{X}) = P(\tilde{X} | \Theta)P(\Theta)/P(\tilde{X})$$

$$\log P(\Theta | \tilde{X}) = \log P(\tilde{X} | \Theta) + \log P(\Theta)$$

在参数估计时都是最大似然；
在进行概率值推断的时候，最大后验需要加上参数的先验分布



- MAP 最大后验

- 给定样本和参数的先验分布，求最可能出现的参数

$$\hat{\Theta}_{MAP} = \arg \max_{\Theta} p(\Theta | X, \alpha)$$

- 后验分布：同时考虑观测样本和参数的先验分布

$$\begin{aligned}\hat{\Theta} &= \arg \max_{\Theta} p(\Theta | X, \alpha) \\ &= \arg \max_{\Theta} \frac{p(X | \Theta)p(\Theta | \alpha)}{p(X)} \\ &\propto \arg \max_{\Theta} p(X | \Theta)p(\Theta | \alpha) \\ &= \arg \max_{i=1} \left\{ \underbrace{\sum_{i=1}^n \log p(x_i | \Theta)}_{\text{最大似然}} + \log \underbrace{p(\Theta | \alpha)}_{\text{先验分布}} \right\}\end{aligned}$$



- 隐变量即变量是不可观测的



假设参数 z 的分布的参数是 θ

- 最大似然法：当不存在隐变量

➤ 直接利用求导为零求解

$$L(x_1, x_2, x_3, \dots, x_N; \Theta) = \prod_{i=1}^N p(x_i | \Theta)$$

$$\ln L(x_1, x_2, x_3, \dots, x_N; \Theta) = \sum_{i=1}^N \ln(p(x_i | \Theta))$$

$$\frac{\partial (\ln L(x_1, x_2, x_3, \dots, x_N; \Theta))}{\partial \Theta} = 0$$



点估计

$$p(x | \Theta) = \int p(x | z)p(z | \Theta)dz$$



• EM算法推导

- 加入隐变量之后的似然

相除

$$p(X, z | \Theta) = \frac{p(X, z, \Theta)}{p(\Theta)}$$

$$p(z | X, \Theta) = \frac{p(z, X, \Theta)}{p(X, \Theta)}$$

条件概率

$$\frac{p(X, z | \Theta)}{p(z | X, \Theta)} = \frac{\frac{p(X, z, \Theta)}{p(\Theta)}}{\frac{p(z, X, \Theta)}{p(X, \Theta)}} = \frac{p(X, \Theta)}{p(\Theta)} = p(X | \Theta)$$

引入任意分布 $z \sim q(z)$

两边同乘 $q(z)$

$$\begin{aligned} \log p(X | \Theta) &= [\log p(X, z | \Theta) - \log q(z)] - [\log p(z | X, \Theta) - \log q(z)] \\ &= \log \frac{p(X, z | \Theta)}{q(z)} - \log \frac{p(z | X, \Theta)}{q(z)} \end{aligned}$$

$$q(z) \log p(X | \Theta) = q(z) \cdot \log \frac{p(X, z | \Theta)}{q(z)} - q(z) \cdot \log \frac{p(z | X, \Theta)}{q(z)}$$

两边对 $q(z)$ 积分

$$\begin{aligned} \int q(z) \log p(X | \Theta) dz &= \int q(z) \cdot \log \frac{p(X, z | \Theta)}{q(z)} dz - \int q(z) \cdot \log \frac{p(z | X, \Theta)}{q(z)} dz \\ &= \int q(z) \cdot \log \frac{p(X, z | \Theta)}{q(z)} dz + \int q(z) \cdot \log \frac{q(z)}{p(z | X, \Theta)} dz \end{aligned}$$

得到KL散度

对任意 $q(z)$ 左边积分为1

$$\log p(X | \Theta) = \underbrace{E_{z \sim q(z)} \left[\log \frac{p(X, z | \Theta)}{q(z)} \right]}_{\text{Lower Bound } B} + \underbrace{KL[q(z) \| p(z | X, \Theta)]}_{\geq 0}$$

点估计



$$\log p(x | \Theta) = \underbrace{E_{z \sim q(z)} \left[\log \frac{p(x, z | \Theta)}{q(z)} \right]}_{\text{Lower Bound } B} + \underbrace{KL[q(z) \| p(z | x, \Theta)]}_{\geq 0}$$

- 难点：既需要优化参数，也需要优化分布 $q(z)$

$$B \leftarrow B_{q(z)}(\Theta), KL \leftarrow KL_{\Theta}(q(z))$$

- E步: $\min_{q(z)} KL[q(z) \| p(z | X, \theta)]$
- M步: $\max_{\Theta} B = E_{z \sim q(z)} \left[\log \frac{p(X, z | \Theta)}{q(z)} \right] = E_{z \sim q(z)} [\log p(x | z, \Theta)] - KL[q(z) \| p(z)]$

这个期望怎么算呢？

- 直接采样：

$$E_{z \sim q(z)} [\log p(x | z, \Theta)] = \frac{1}{N} \sum_{z_i} \log p(x | z_i, \Theta)$$

- 变分法（在后面的推断部分会讲到）

变分推断



$$\begin{aligned} & \bullet \text{ E步: } \min_{q(z)} KL[q(z) \| p(z | X, \theta)] \\ & \bullet \text{ M步: } \max_{\Theta} B = E_{z \sim q(z)} \left[\log \frac{p(X, z | \Theta)}{q(z)} \right] = \boxed{E_{z \sim q(z)} [\log p(x | z, \Theta)]} - KL[q(z) \| p(z)] \end{aligned}$$

- 如何定义 $q(z)$ 并在E步中进行求解?
- 如何处理M step中的积分项?

方法1: MCMC求解

- 思路: 不对后验分布 $p(z|x, \theta)$ 做近似, 而是直接通过数值模拟出期望值, 代入到目标中
- E Step: 直接从后验 $p(z|x, \theta)$ 中采样 L 个样本 $z^{(l)}$
 $q(z) = p(z|x, \theta)$
- M Step: 对积分项采样模拟, 梯度下降更新 θ

$$E_{z \sim q(z)} [\log p(x|z, \theta)] \approx \frac{1}{L} \sum_{l=1}^L \log p(x|z^{(l)}, \theta) \text{ , where } z^{(l)} \sim q(z)$$

变分推断



方法2：变分推断求解

- 思想：对后验分布 $p(z|x, \theta)$ 做近似，然后求解近似分布的参数
- 给分布 $q(z)$ 引入参数 ϕ 得到 $q(z|\phi)$ ，E中优化 ϕ ，M中优化 θ （对于积分项直接数学求解）

1. **E Step**: 通过参数 ϕ 找到一个分布 $q(z|\phi)$ 使得这个分布与 z 的后验分布 $p(z|x, \theta)$ 之间的 KL 散度最小，即：

$$\min_{\phi} KL[q(z|\phi) \| p(z|x, \theta)]$$

后验分布难以求解

2. **M Step**: 基于优化的参数 ϕ 得到似然函数的下界 B ，通过优化参数 θ 从而最大化这个下界，即：

$$\max_{\theta} B = E_{z \sim q(z|\phi)} \left[\log \frac{p(x, z|\theta)}{q(z|\phi)} \right]$$

变分推断



- 变分推断怎么处理棘手的后验分布？

- 原EM优化：

$$\log p(x|\theta) = \underbrace{E_{z \sim q(z|\phi)} [\log p(x|z, \theta)] - KL[q(z|\phi) \| p(z)]}_{\max_{\theta}} + \underbrace{KL[q(z|\phi) \| p(z|x, \theta)]}_{\min_{\phi}}$$

左项，即B (M Step) 右项 (E Step)

存在关系：

右项 = -左项 + $\log p(x|\theta)$

通过优化 ϕ 增大左项
-> 减小右项

- 新的EM优化方法：

$$\log p(x|\theta) = \underbrace{E_{z \sim q(z|\phi)} [\log p(x|z, \theta)] - KL[q(z|\phi) \| p(z)]}_{\max_{\theta} \max_{\phi}} + KL[q(z|\phi) \| p(z|x, \theta)]$$

通常需要设计合理的 $q(z|\phi)$ ，
使得积分项可解。

- E Step: 优化参数 ϕ 以最大化左项
- M Step: 代入E Step的 ϕ ，优化参数 θ 以最大化左一项（左二与 θ 无关）
- EM算法中参数更新的方法为：求解相应项的导数的零点

缺点：

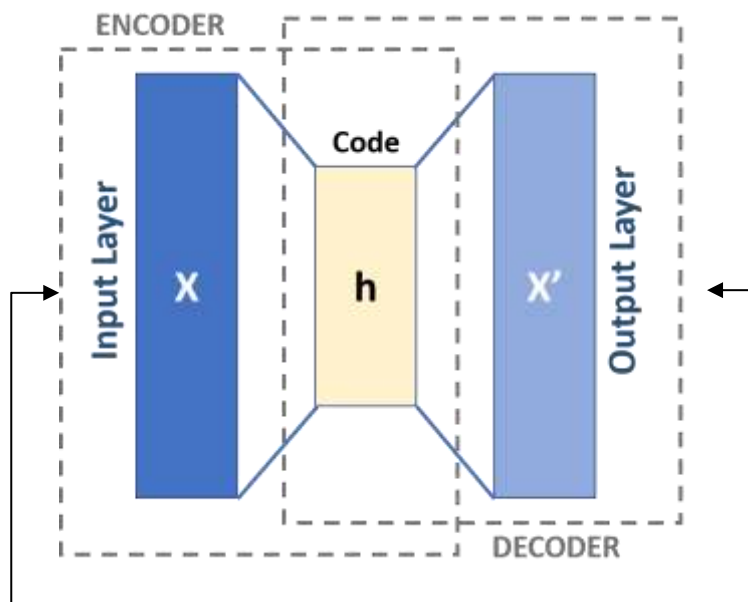
1. 分布 $q(z|\phi)$ 的设计有时很难
2. 效果易受近似分布形式的影响

变分推断



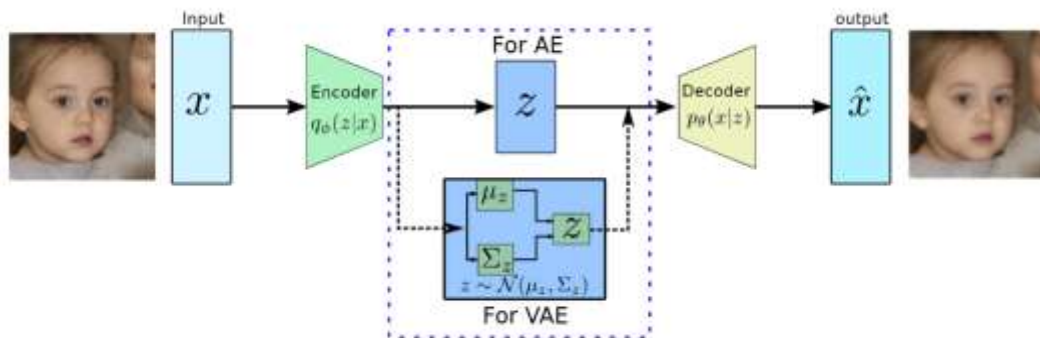
实例：Variational Auto-Encoder (VAE)

传统AE



Reconstruction Loss

VAE



变分推断



$$\log p(x|\theta) = \underbrace{E_{z \sim q(z|\phi)} [\log p(x|z, \theta)] - KL[q(z|\phi) \| p(z)]}_{\max_{\theta} \max_{\phi}} + KL[q(z|\phi) \| p(z|x, \theta)]$$

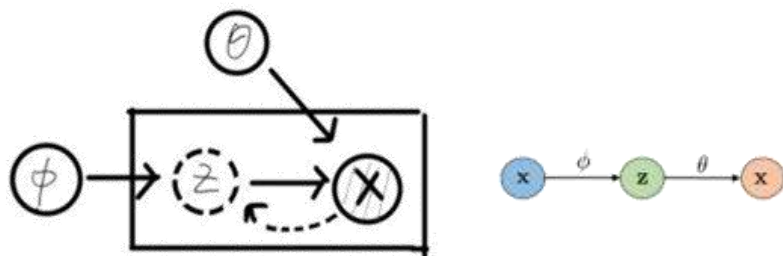
通常需要设计合理的 $q(z|\phi)$,
使得积分项可解。

- E Step: 优化参数 ϕ 以最大化左项
- M Step: 代入E Step的 ϕ , 优化参数 θ 以最大化左一项 (左二与 θ 无关)
- EM算法中参数更新的方法为: 求解相应项的导数的零点

缺点:

1. 分布 $q(z|\phi)$ 的设计有时很难
2. 效果易受近似分布形式的影响

• VAE概率图模型的设计



- 缩小解空间, 将变分分布设计为 $q(z|x, \phi)$
- 把先验 $p(x|z, \theta)$ 变形为 $p(x|f_{\theta}(z))$
- 把先验 $q(z|x, \phi)$ 变形为 $q(z|g_{\phi}(x))$

$f_{\theta}(\cdot)$ 可看作decoder
 $g_{\phi}(\cdot)$ 可看作encoder

变分推断



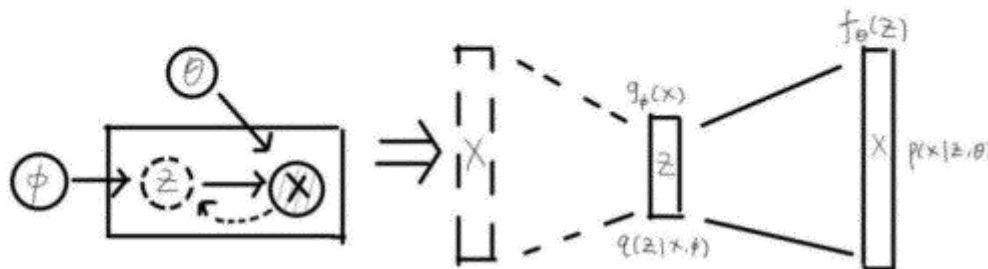
VAE概率图模型的形式

- 在前文的变分推断求解中，最大化似然概率 $\log p(x|\theta)$ 等价于优化下式红框：

$$\log p(x|\theta) = \underbrace{E_{z \sim q(z|\phi)} [\log p(x|z, \theta)] - KL[q(z|\phi) \| p(z)]}_{\max_{\theta} \max_{\phi}} + KL[q(z|\phi) \| p(z|x, \theta)]$$

- VAE的方程：

$$\underbrace{\log p(x|\theta)}_{\text{likelihood}} - \underbrace{KL[q(z|g_{\phi}(x)) \| p(z|x, \theta)]}_{\text{encoder}} = \underbrace{E_{q(z|g_{\phi}(x))} [\log p(x|f_{\theta}(z))]}_{\text{decoder}} - \underbrace{KL[q(z|g_{\phi}(x)) \| p(z)]}_{\text{encoder prior}}$$



x 为观测变量， z 为隐藏变量，
模型参数为 θ, ϕ

求解目标：为了最大化似然，
通过 θ, ϕ 直接优化右式的下界。

VAE概率图模型的求解

- 优化目标为等号右侧

$$\log \underbrace{p(x|\theta)}_{\text{likelihood}} - KL \left[\underbrace{q(z|g_\phi(x))}_{\text{encoder}} \parallel \underbrace{p(z|x, \theta)}_{\text{posterior}} \right] = \overbrace{E_{q(z|g_\phi(x))} [\log p(x|f_\theta(z))]}^{\mathcal{L}_2(\theta, \phi|x^{(i)})} - \overbrace{KL \left[\underbrace{q(z|g_\phi(x))}_{\text{encoder}} \parallel \underbrace{p(z)}_{\text{prior}} \right]}^{\mathcal{L}_1(\phi|x^{(i)})}$$

- 对于数据点 $x^{(i)}$ ，需最大化“损失函数”，即 $\mathcal{L}(\theta, \phi|x^{(i)}) = \mathcal{L}_1(\phi|x^{(i)}) + \mathcal{L}_2(\theta, \phi|x^{(i)})$

- 更新方式为随机梯度“下降”

$$\{\theta, \phi\} = \{\theta, \phi\} + \lambda \cdot \frac{\mathcal{L}(\theta, \phi|x^{(i)})}{\partial \{\theta, \phi\}}$$

变分推断



- 优化目标为等号右侧

$$\log \underbrace{p(x|\theta)}_{\text{likelihood}} - KL \left[\underbrace{q(z|g_\phi(x))}_{\text{encoder}} \parallel \underbrace{p(z|x, \theta)}_{\text{posterior}} \right] = \overbrace{E_{q(z|g_\phi(x))} [\log p(x|f_\theta(z))]}^{\text{reconstruction error } \mathcal{L}_2(\theta, \phi|x^{(i)})} - \overbrace{KL \left[\underbrace{q(z|g_\phi(x))}_{\text{encoder}} \parallel \underbrace{p(z)}_{\text{prior}} \right]}^{\text{regularization term } \mathcal{L}_1(\phi|x^{(i)})}$$

- 对于数据点 $x^{(i)}$ ，需最大化“损失函数”，即 $\mathcal{L}(\theta, \phi|x^{(i)}) = \mathcal{L}_1(\phi|x^{(i)}) + \mathcal{L}_2(\theta, \phi|x^{(i)})$

- 更新方式为随机梯度“下降”：

$$\{\theta, \phi\} = \{\theta, \phi\} + \lambda \cdot \frac{\mathcal{L}(\theta, \phi|x^{(i)})}{\partial \{\theta, \phi\}}$$

- $\mathcal{L}_1$ 项KL散度容易计算（例如假设 $q(z|g_\phi(x))$ 和 $p(z)$ 都是高斯分布），导数也易求得

- 对于 $\mathcal{L}_2$ 项梯度的求解：

- (×) 在变分推断求解中的积分求解，容易受近似分布形式的影响，难度较大
- (×) 在采样求解中的采样+数值模拟，存在效率问题，且采样结果丢失分布的参数信息：

$$\frac{\partial E_{q(z|g_\phi(x))} [\log p(x|f_\theta(z))]}{\partial \phi} \approx \frac{\frac{1}{L} \sum_{l=1}^L \partial \log p(x|f_\theta(z^{(l)}))}{\partial \phi} = 0, \text{ where } z^{(l)} \sim q(z|g_\phi(x)) \quad \Rightarrow \text{reparameterization}$$

$$\begin{aligned} D_{KL}(q(z|x)||p(z)) &= \int q(z|x) \log \frac{q(z|x)}{p(z)} dz \\ &= \int q(z|x) \{ \log q(z|x) - \log p(z) \} dz \\ &= \int q(z|x) \log q(z|x) dz - \int q(z|x) \log p(z) dz \\ &= \left(-\frac{J}{2} \log(2\pi) - \frac{1}{2} \sum_{j=1}^J (\log \sigma_j^2 + 1) \right) - \left(-\frac{J}{2} \log(2\pi) - \frac{1}{2} \sum_{j=1}^J (\sigma_j^2 + \mu_j^2) \right) \\ &= -\frac{1}{2} \sum_{j=1}^J \left(1 + \log \sigma_j^2 - \sigma_j^2 - \mu_j^2 \right) \end{aligned} \quad (5)$$

变分推断



SGVB求解

$$\mathcal{L}(\theta, \phi | x^{(i)}) = \underbrace{E_{q(z|g_\phi(x))} [\log p(x|f_\theta(z))]}_{\text{decoder}} - \underbrace{KL[q(z|g_\phi(x)) \parallel p(z)]}_{\text{encoder prior}}$$

$\mathcal{L}_2(\theta, \phi | x^{(i)}) + \mathcal{L}_1(\phi | x^{(i)})$

- 采样丢失了变分分布 $z \sim q(z|g_\phi(x))$ 的参数 ϕ 的梯度，故设计对 z 进行 **reparameterization**:

$$z^{(l)} = \tilde{g}_\phi(\epsilon^{(l)}, x), \text{ where } \epsilon^{(l)} \sim p(\epsilon)$$

- 对于一个微小噪声变量 $\epsilon^{(l)} \sim p(\epsilon)$ ，基于一个确定性的函数 $\tilde{g}_\phi(\epsilon, x)$ 实现与 $z^{(l)}$ 的一一对应
- 把 $z = \tilde{g}_\phi(\epsilon, x)$ 代入目标函数

变分推断



- 采样丢失了变分分布 $z \sim q(z|g_\phi(x))$ 的参数 ϕ 的梯度，故设计对 z 进行reparameterization:

$$z^{(l)} = \tilde{g}_\phi(\epsilon^{(l)}, x), \text{ where } \epsilon^{(l)} \sim p(\epsilon)$$

- 对于一个微小噪声变量 $\epsilon^{(l)} \sim p(\epsilon)$ ，基于一个确定性的函数 $\tilde{g}_\phi(\epsilon, x)$ 实现与 $z^{(l)}$ 的一一对应
- 把 $z = \tilde{g}_\phi(\epsilon, x)$ 代入 $\mathcal{L}_2(\theta, \phi|x^{(i)})$ ，有：

$$\begin{aligned}\mathcal{L}_2(\theta, \phi|x^{(i)}) &= E_{q(z|g_\phi(x))}[\log p(x|f_\theta(z))] \\ &= E_{p(\epsilon)}[\log p(x|f_\theta(\tilde{g}_\phi(\epsilon, x)))] \\ &\approx \frac{1}{L} \sum_l \log p(x|f_\theta(\tilde{g}_\phi(\epsilon^{(l)}, x)))\end{aligned}$$

间接地从 $p(\epsilon)$ 而非 $q(z|g_\phi(x))$ 中采样，
原先分布中的参数 ϕ 转移到了期望的内容中。
这时数值模拟的 $\mathcal{L}_2$ ，导数可求解。

Algorithm 1 AEVB 对单数据点更新算法

Initialize parameters θ, ϕ

repeat

Draw a datapoint $x^{(i)}$

Draw noise samples $\epsilon^{(l)} \sim p(\epsilon)$

Update θ with:

$$\theta = \theta + \lambda \frac{1}{L} \sum_{l=1}^L \frac{\partial \log p(x^{(i)}|f_\theta(\tilde{g}_\phi(\epsilon^{(l)}, x^{(i)})))}{\partial \theta}$$

Update ϕ with:

$$\phi = \phi + \lambda \cdot \left[\frac{\partial \mathcal{L}_1(\phi|x^{(i)})}{\partial \phi} + \frac{1}{L} \sum_{l=1}^L \frac{\partial \log p(x^{(i)}|f_\theta(\tilde{g}_\phi(\epsilon^{(l)}, x^{(i)})))}{\partial \phi} \right]$$

until convergence

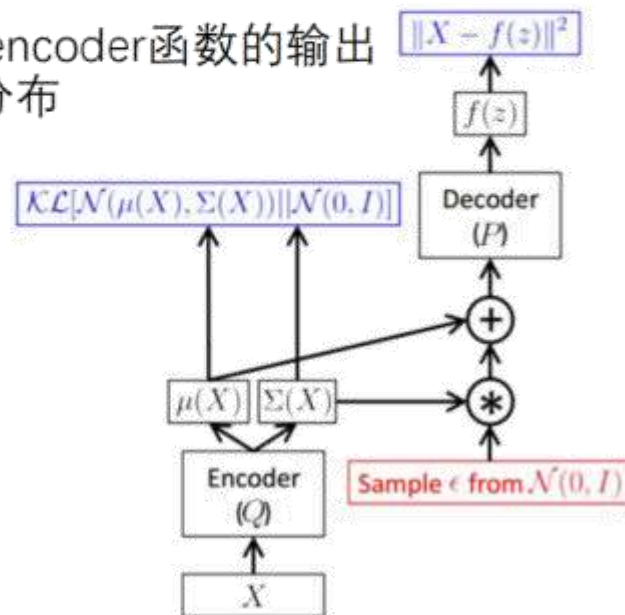
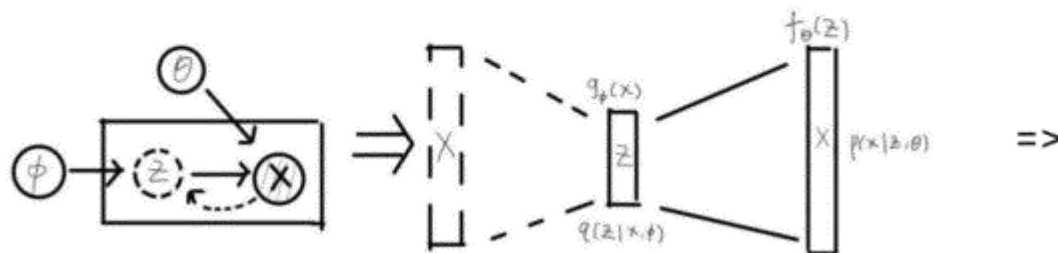
变分推断

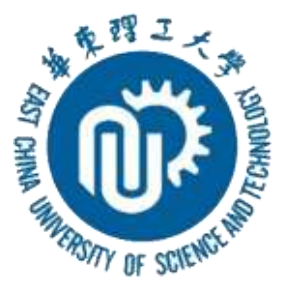


VAE的神经网络实现

$$\mathcal{L}(\theta, \phi | x^{(i)}) \approx \underbrace{\frac{1}{L} \sum_l \log p(x | f_\theta(\tilde{g}_\phi(\epsilon^{(l)}, x)))}_{\text{reconstruction error}} - \underbrace{KL[q(z | g_\phi(x)) || p(z)]}_{\text{regularization term}}$$

- 假设decoder函数的输出 $p(x | f_\theta(x))$ 服从高斯分布，encoder函数的输出 $q(z | g_\phi(x))$ 服从高斯分布，隐变量 z 的先验服从高斯分布
- 基于SGVB，映射函数设计为 $\tilde{g}_\phi(\epsilon, x) = \mu + \sigma \cdot \epsilon$





深度生成模型



生成模型

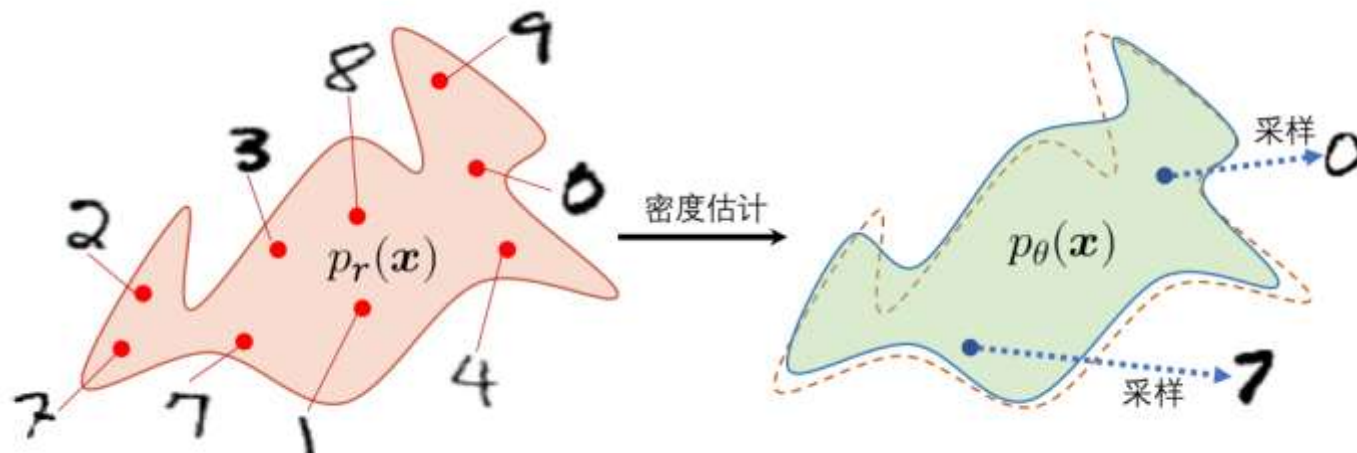


生成模型：一系列用于随机生成可观测数据的模型

- 生成模型包含两个步骤：

密度估计

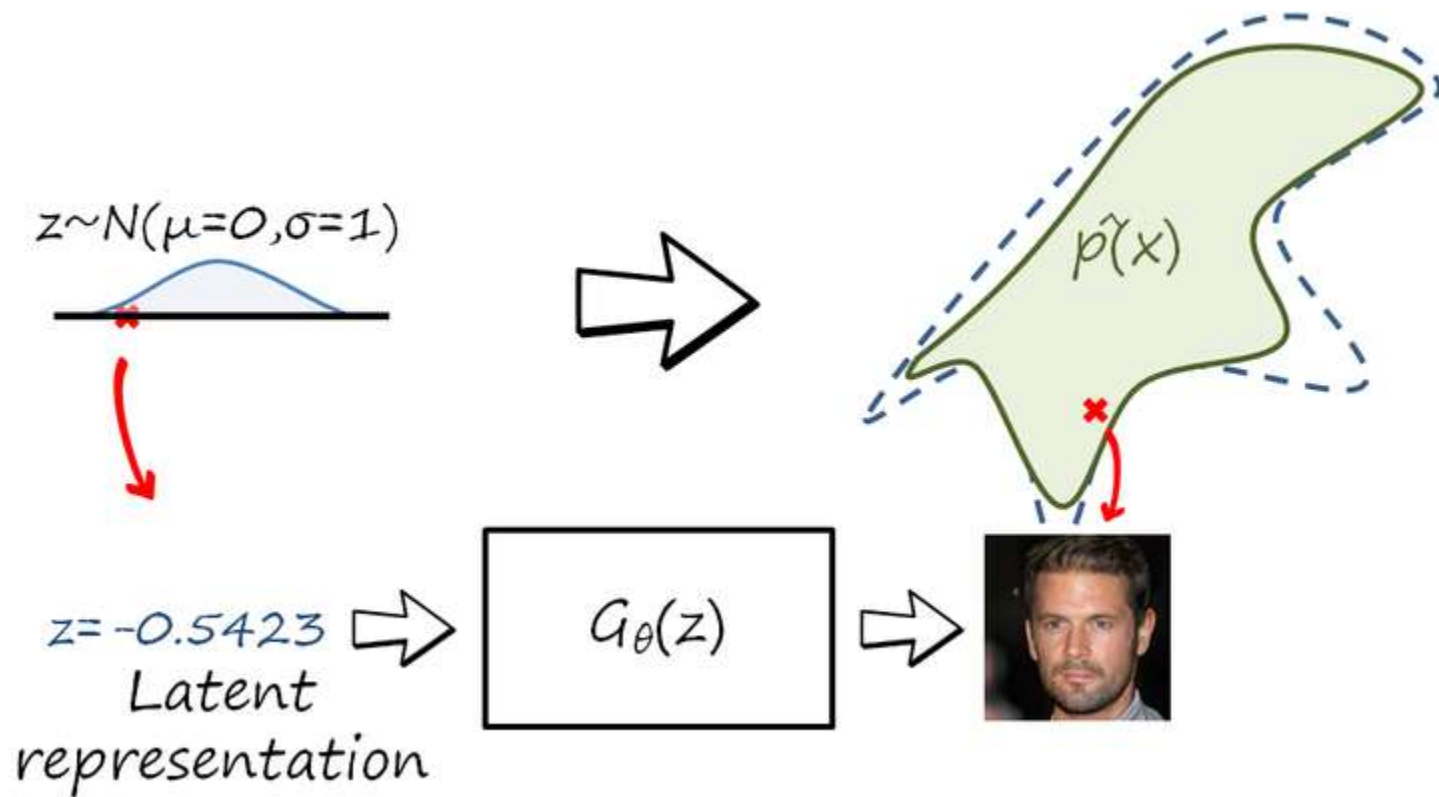
采样



生成模型



- 生成数据的另一种思路



深度生成模型



深度生成模型： 是利用神经网络构建生成模型。

- 变分自编码器 (Variational Autoencoder, VAE)
[Kingma and Welling, 2013, Rezende et al., 2014]
- 生成对抗网络 (Generative Adversarial Network, GAN)
[Goodfellow et al., 2014]
- 扩散模型 (Diffusion Model, DM)
[Jonathan Ho et al., 2020]

MinMax Game



判别网络

$$\max_{\phi} \mathbb{E}_{\mathbf{x} \sim p_r(\mathbf{x})} \left[\log D(\mathbf{x}; \phi) \right] + \mathbb{E}_{\mathbf{z} \sim p(\mathbf{z})} \left[\log(1 - D(G(\mathbf{z}; \theta); \phi)) \right]$$

生成网络

$$\max_{\theta} \left(\mathbb{E}_{\mathbf{z} \sim p(\mathbf{z})} \left[\log D(G(\mathbf{z}; \theta); \phi) \right] \right)$$

判别网络

$$\min_{\theta} \max_{\phi} \left(\mathbb{E}_{\mathbf{x} \sim p_r(\mathbf{x})} \left[\log D(\mathbf{x}; \phi) \right] + \mathbb{E}_{\mathbf{z} \sim p(\mathbf{z})} \left[\log(1 - D(G(\mathbf{z}; \theta); \phi)) \right] \right)$$

模型分析



- 假设 $p_r(x)$ 和 $p_\theta(x)$ 已知, 则最优的判别器为

$$D^*(\mathbf{x}) = \frac{p_r(\mathbf{x})}{p_r(\mathbf{x}) + p_\theta(\mathbf{x})}$$

- 目标函数为

$$\begin{aligned}\mathcal{L}(G|D^*) &= \mathbb{E}_{\mathbf{x} \sim p_r(\mathbf{x})} \left[\log D^*(\mathbf{x}) \right] + \mathbb{E}_{\mathbf{x} \sim p_\theta(\mathbf{x})} \left[\log(1 - D^*(\mathbf{x})) \right] \\ &= \mathbb{E}_{\mathbf{x} \sim p_r(\mathbf{x})} \left[\log \frac{p_r(\mathbf{x})}{p_r(\mathbf{x}) + p_\theta(\mathbf{x})} \right] + \mathbb{E}_{\mathbf{x} \sim p_\theta(\mathbf{x})} \left[\log \frac{p_\theta(\mathbf{x})}{p_r(\mathbf{x}) + p_\theta(\mathbf{x})} \right] \\ &= D_{\text{KL}}(p_r \| p_a) + D_{\text{KL}}(p_\theta \| p_a) - 2 \log 2 \\ &= 2D_{\text{JS}}(p_r \| p_\theta) - 2 \log 2,\end{aligned}$$

$$D_{\text{JS}}(p \| q) = \frac{1}{2} D_{\text{KL}}(p \| m) + \frac{1}{2} D_{\text{KL}}(q \| m)$$

$$m = \frac{1}{2}(p + q)$$

模型坍塌：生成网络的“错误”目标



- 生成网络的目标函数

$$\begin{aligned}\mathcal{L}'(G|D^*) &= \mathbb{E}_{\mathbf{x} \sim p_\theta(\mathbf{x})} \left[\log D^*(\mathbf{x}) \right] \\ &= \mathbb{E}_{\mathbf{x} \sim p_\theta(\mathbf{x})} \left[\log \frac{p_r(\mathbf{x})}{p_r(\mathbf{x}) + p_\theta(\mathbf{x})} \cdot \frac{p_\theta(\mathbf{x})}{p_\theta(\mathbf{x})} \right] \\ &= -\mathbb{E}_{\mathbf{x} \sim p_\theta(\mathbf{x})} \left[\log \frac{p_\theta(\mathbf{x})}{p_r(\mathbf{x})} \right] + \mathbb{E}_{\mathbf{x} \sim p_\theta(\mathbf{x})} \left[\log \frac{p_\theta(\mathbf{x})}{p_r(\mathbf{x}) + p_\theta(\mathbf{x})} \right] \\ &= -D_{\text{KL}}(p_\theta \| p_r) + \mathbb{E}_{\mathbf{x} \sim p_\theta(\mathbf{x})} \left[\log (1 - D^*(\mathbf{x})) \right] \\ &= -D_{\text{KL}}(p_\theta \| p_r) + 2D_{\text{JS}}(p_r \| p_\theta) - 2\log 2 - \mathbb{E}_{\mathbf{x} \sim p_r(\mathbf{x})} \left[\log D^*(\mathbf{x}) \right]\end{aligned}$$

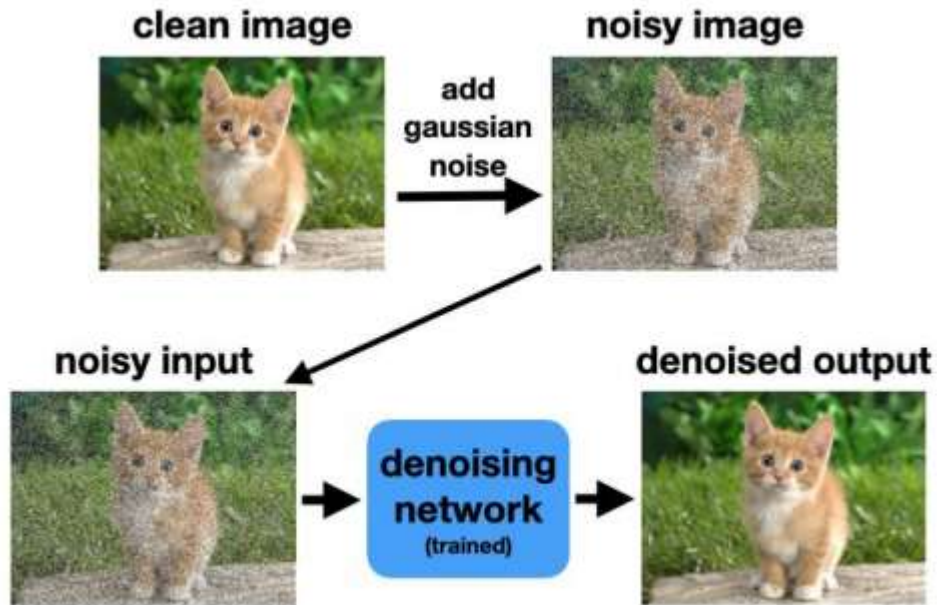
- 其中后两项和生成网络无关，因此

$$\arg \max_{\theta} \mathcal{L}'(G|D^*) = \arg \min_{\theta} D_{\text{KL}}(p_\theta \| p_r) - 2D_{\text{JS}}(p_r \| p_\theta)$$

Diffusion Model



Diffusion最初应用：图像去噪



Denoising loss for training diffusion models

$$\sum_{\text{images}} \left\| \underbrace{\text{denoising network (trained)} \left(\text{noisy image} \right)}_{\text{Denoised output image}} - \text{ground truth image} \right\|_1$$

Diffusion Model



Diffusion最初应用：图像去噪

这是不是意味着给定一张完全是噪声的图像也可以通过“去噪”恢复出完整图像

Image generation via extreme denoising

random
noise array



**denoising
network**



Behold!
An image!

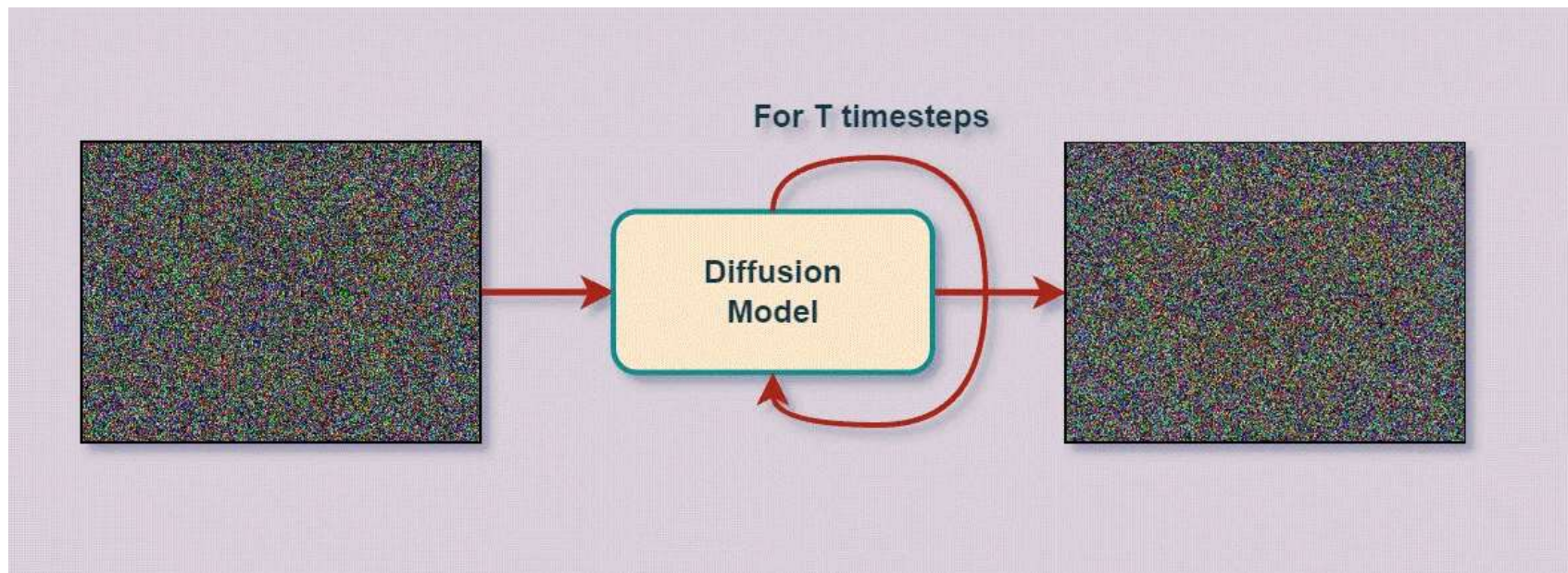


Diffusion Model



Diffusion最初应用：图像去噪

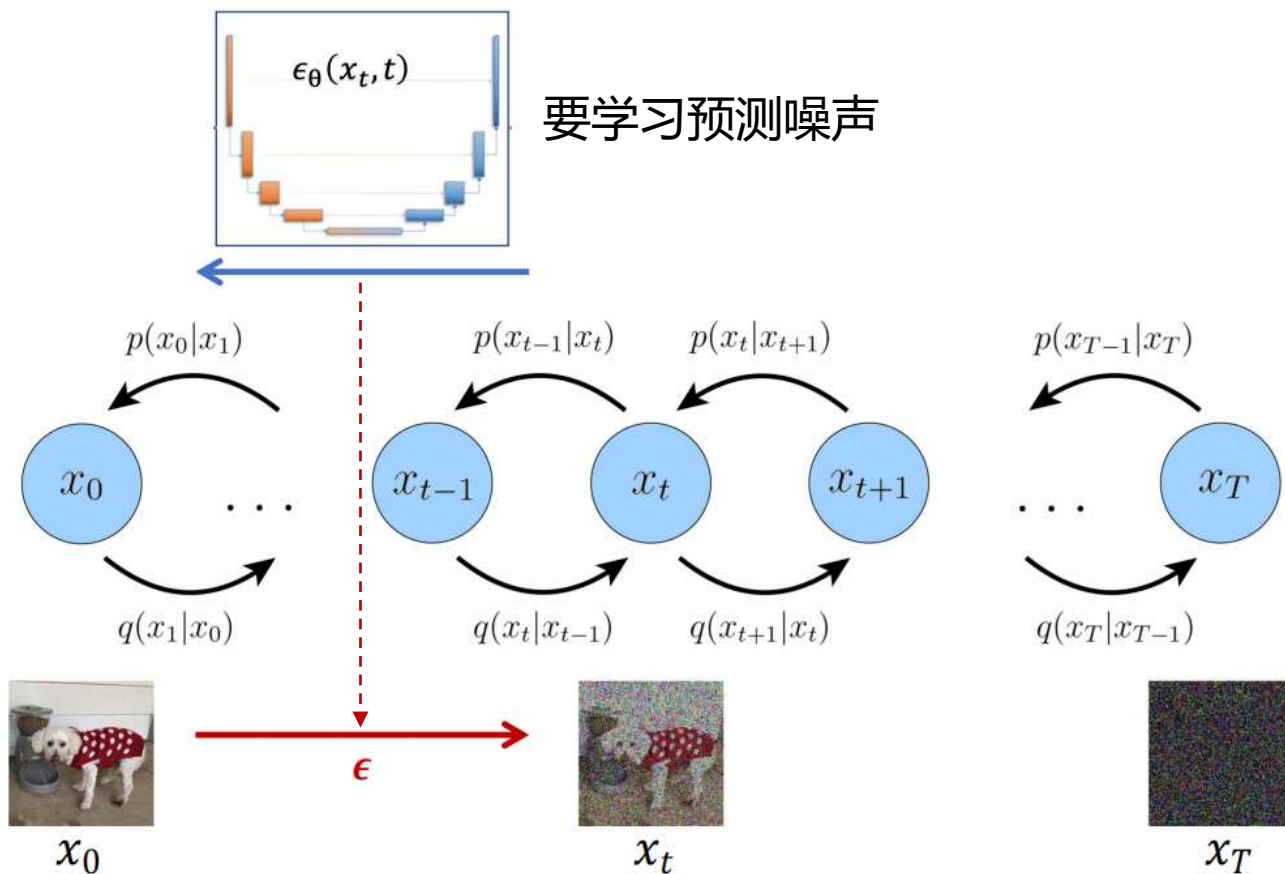
这是不是意味着给定一张完全是噪声的图像也可以通过“去噪”恢复出完整图像



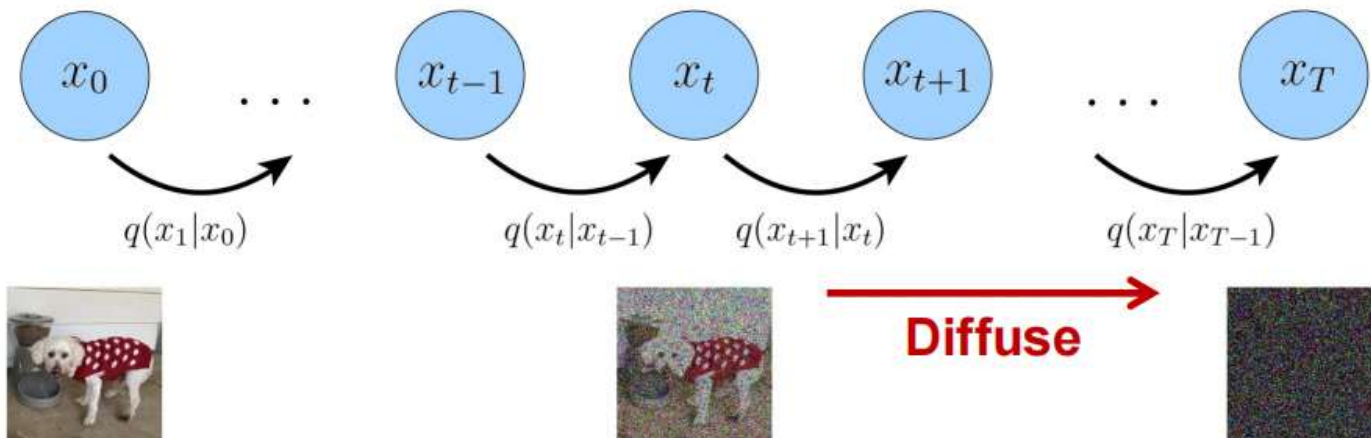
Diffusion Model



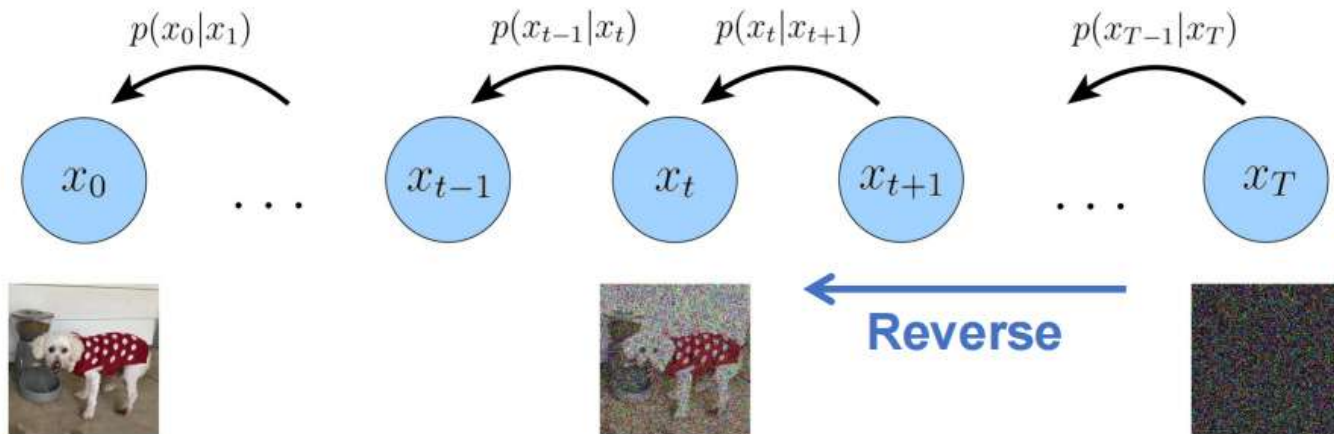
Diffusion做图像生成的动机：逐步加入特定噪声，再逐步去噪



Diffusion Model



随机给定一个噪声，噪声预测网络预测扩散阶段每一步添加的噪声，实现反向去噪



Diffusion Model



Algorithm 1 Training

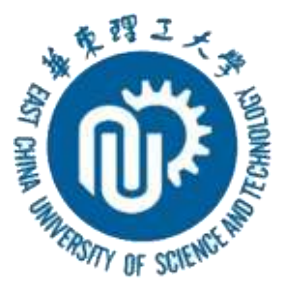
```
1: repeat
2:    $\mathbf{x}_0 \sim q(\mathbf{x}_0)$ 
3:    $t \sim \text{Uniform}(\{1, \dots, T\})$ 
4:    $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ 
5:   Take gradient descent step on
        $\nabla_{\theta} \|\epsilon - \epsilon_{\theta}(\sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, t)\|^2$ 
6: until converged
```

基于训练集训练噪声预测网络

Algorithm 2 Sampling

```
1:  $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ 
2: for  $t = T, \dots, 1$  do
3:    $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$  if  $t > 1$ , else  $\mathbf{z} = \mathbf{0}$ 
4:    $\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left( \mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}} \epsilon_{\theta}(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}$ 
5: end for
6: return  $\mathbf{x}_0$ 
```

给定一个起始点，预测其每一步“被添加的”噪声，实现去噪



深度强化学习





智能体 (Agent)

感知外界环境的**状态** (State) 和奖励**反馈** (Reward), 并进行学习和决策。智能体的**决策**功能是指根据外界环境的状态来做出不同的**动作** (Action), 而**学习**功能是指根据外界环境的奖励来调整策略。

环境 (Environment)

智能体外部的所有事物, 并受智能体动作的影响而改变其状态, 并反馈给智能体相应的奖励。

强化学习中的基本要素



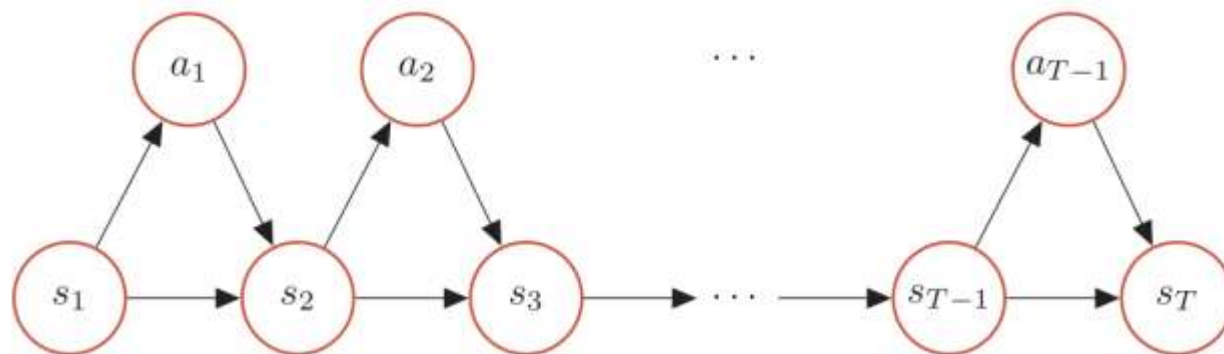
- 环境的**状态集合**： S
- 智能体的**动作集合**： A
- **策略**： $\pi(a|s)$ ，即智能体根据环境状态 s 来决定下一步动作 a 的函数
- **状态转移概率**： $p(s'|s, a)$ ，即智能体根据当前状态 s 做出一个动作 a 之后，下一个时刻环境处于不同状态 s' 的概率；
- **即时奖励**： $r(s, a, s')$ ，即智能体根据当前状态做出一个动作之后，环境会反馈给智能体一个奖励，这个奖励和动作之后下一个时刻的状态有关。

策略 $\pi(a|s)$



- 马尔可夫决策过程的一个**轨迹** (trajectory)

$$\tau = s_0, a_0, s_1, r_1, a_1, \dots, s_{T-1}, a_{T-1}, s_T, r_T$$



- τ 的概率

$$p(\tau) = p(s_0, a_0, s_1, a_1, \dots),$$

$$= p(s_0) \prod_{t=0}^{T-1} \pi(a_t|s_t) p(s_{t+1}|s_t, a_t).$$

总回报



- 给定策略 $\pi(a|s)$ ，智能体和环境一次交互过程的轨迹 τ 所收到的累积奖励为**总回报** (return)

$$G(\tau) = \sum_{t=0}^{T-1} \gamma^t r_{t+1}$$

- $\gamma \in [0,1]$ 是**折扣率**。当 γ 接近于0时，智能体更在意短期回报；而当 γ 接近于1时，长期回报变得更重要。
- 环境中有一个或多个特殊的终止状态 (terminal state) 。

强化学习目标函数

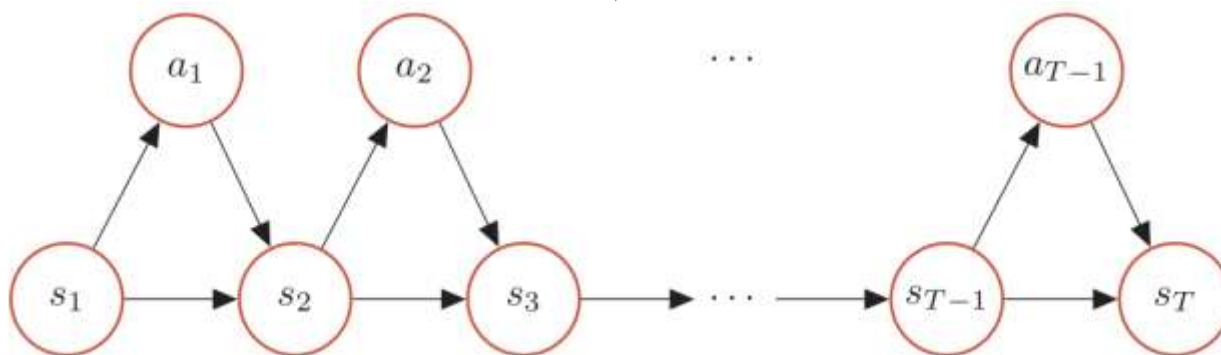


- 强化学习的目标是学习到一个策略 $\pi_{\theta}(a|s)$ 来**最大化期望回报** (expected return) 。

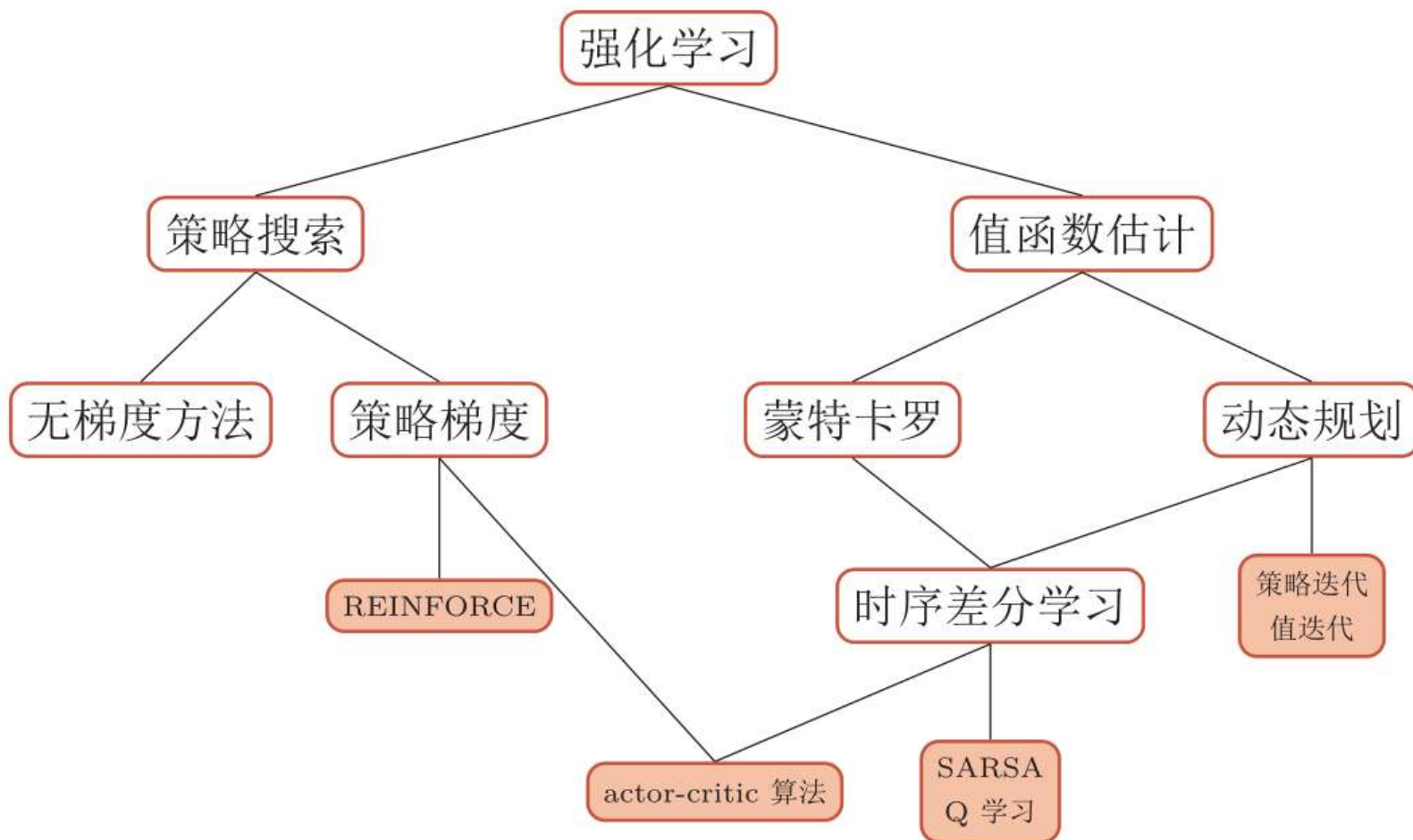
$$J(\theta) = \mathbb{E}_{\tau \sim p_{\theta}(\tau)}[G(\tau)] = \mathbb{E}_{\tau \sim p_{\theta}(\tau)} \left[\sum_{t=0}^{T-1} \gamma^t r_{t+1} \right]$$

- θ 为策略函数的参数

希望找到一个决策过程能使得期望回报最大



强化学习的算法



状态值函数



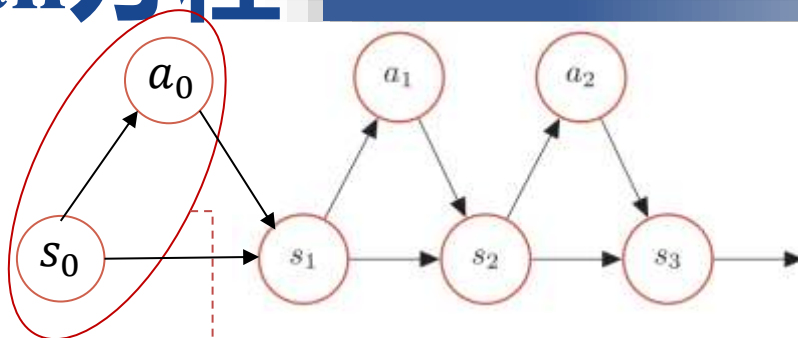
- 一个策略 π 期望回报可以分解为

$$\begin{aligned}\mathbb{E}_{\tau \sim p(\tau)}[G(\tau)] &= \mathbb{E}_{s \sim p(s_0)} \left[\mathbb{E}_{\tau \sim p(\tau)} \left[\sum_{t=0}^{T-1} \gamma^t r_{t+1} \mid \tau_{s_0} = s \right] \right] \\ &= \mathbb{E}_{s \sim p(s_0)} [V^\pi(s)],\end{aligned}$$

- 值函数**：从状态 s 开始，执行策略 π 得到的期望总回报

$$V^\pi(s) = \mathbb{E}_{\tau \sim p(\tau)} \left[\sum_{t=0}^{T-1} \gamma^t r_{t+1} \mid \tau_{s_0} = s \right]$$

Bellman方程

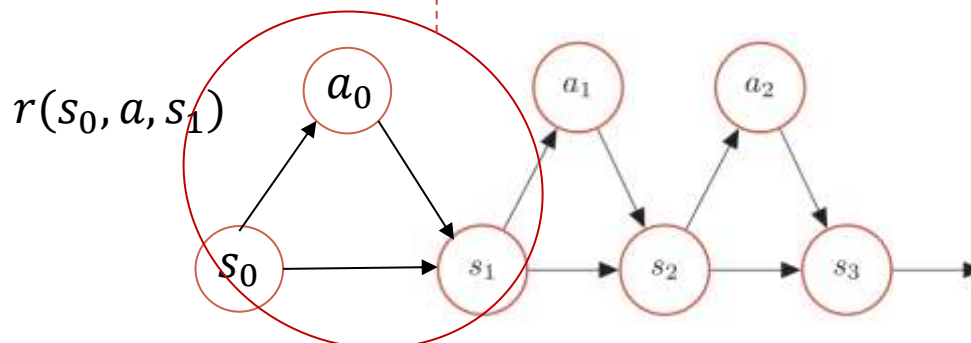


$$V^\pi(s) = \mathbb{E}_{\tau_0:T \sim p(\tau)} \left[r_1 + \gamma \sum_{t=1}^{T-1} \gamma^{t-1} r_{t+1} \mid \tau_{s_0} = s \right]$$

$$= \mathbb{E}_{a \sim \pi(a|s)} \mathbb{E}_{s' \sim p(s'|s,a)} \mathbb{E}_{\tau_1:T \sim p(\tau)} \left[\overset{\text{进一步改写}}{r(s, a, s')} + \gamma \sum_{t=1}^{T-1} \gamma^{t-1} r_{t+1} \mid \tau_{s_1} = s' \right]$$

$$= \mathbb{E}_{a \sim \pi(a|s)} \mathbb{E}_{s' \sim p(s'|s,a)} \left[r(s, a, s') + \gamma \mathbb{E}_{\tau_1:T \sim p(\tau)} \left[\sum_{t=1}^{T-1} \gamma^{t-1} r_{t+1} \mid \tau_{s_1} = s' \right] \right]$$

$$= \mathbb{E}_{a \sim \pi(a|s)} \mathbb{E}_{s' \sim p(s'|s,a)} \left[r(s, a, s') + \gamma V^\pi(s') \right].$$



状态-动作值函数(Q函数)



- **状态-动作值函数**是指初始状态为 s 并进行动作 a ，然后执行策略 π 得到的期望总回报。

$$Q^{\pi}(s, a) = \mathbb{E}_{s' \sim p(s'|s, a)} [r(s, a, s') + \gamma V^{\pi}(s')]$$

- Q函数的贝尔曼方程

$$Q^{\pi}(s, a) = \mathbb{E}_{s' \sim p(s'|s, a)} \left[r(s, a, s') + \gamma \mathbb{E}_{a' \sim \pi(a'|s')} [Q^{\pi}(s', a')] \right]$$

最优策略



- 最优策略：存在一个最优的策略 π^* ，其在所有状态上的期望回报最大。

$$\forall s, \quad \pi^* = \arg \max_{\pi} V^{\pi}(s). \quad \text{难以实现!}$$

策略改进：

- 值函数可以看作是对策略 π 的评估。
- 如果在状态 s ，有一个动作 a 使得 $Q^{\pi}(s, a) > V^{\pi}(s)$ ，说明执行动作 a 比当前的策略 $\pi(a|s)$ 要好，我们就可以调整参数使得策略 $\pi(a|s)$ 的概率增加。

基于策略函数的深度强化学习



- 可以直接用深度神经网络来表示一个参数化的从状态空间到动作空间的映射函数： $a = \pi_{\theta}(s)$ 。
- 最优的策略是使得在每个状态的总回报最大的策略，因此策略搜索的目标函数为

$$\mathcal{J}(\theta) = \mathbb{E}_{\tau \sim p_{\theta}(\tau)}[G(\tau)] = \mathbb{E}_{\tau \sim p_{\theta}(\tau)}\left[\sum_{t=0}^{T-1} \gamma^t r_{t+1}\right]$$



- 策略搜索是通过寻找参数 θ 使得目标函数 $J(\theta)$ 最大。
- 梯度上升：

轨迹概率

总回报

$$\begin{aligned}\frac{\partial J(\theta)}{\partial \theta} &= \frac{\partial}{\partial \theta} \int p_{\theta}(\tau) G(\tau) d\tau \quad \tau: \text{轨迹} \\ &= \int \left(\frac{\partial}{\partial \theta} p_{\theta}(\tau) \right) G(\tau) d\tau \\ &= \int p_{\theta}(\tau) \left(\frac{1}{p_{\theta}(\tau)} \frac{\partial}{\partial \theta} p_{\theta}(\tau) \right) G(\tau) d\tau \\ &= \int p_{\theta}(\tau) \left(\frac{\partial}{\partial \theta} \log p_{\theta}(\tau) \right) G(\tau) d\tau \\ &= \mathbb{E}_{\tau \sim p_{\theta}(\tau)} \left[\frac{\partial}{\partial \theta} \log p_{\theta}(\tau) G(\tau) \right],\end{aligned}$$

进一步分解



- 参数 θ 优化的方向是使得总回报 $G(\tau)$ 越大的轨迹 τ 的概率 $p_\theta(\tau)$ 也越大。

$$\begin{aligned}\frac{\partial}{\partial \theta} \log p_\theta(\tau) &= \frac{\partial}{\partial \theta} \log \left(p(s_0) \prod_{t=0}^{T-1} \pi_\theta(a_t|s_t) p(s_{t+1}|s_t, a_t) \right) \\ &= \frac{\partial}{\partial \theta} \left(\log p(s_0) + \sum_{t=0}^{T-1} \log \pi_\theta(a_t|s_t) + \log p(s_{t+1}|s_t, a_t) \right) \\ &= \sum_{t=0}^{T-1} \frac{\partial}{\partial \theta} \log \pi_\theta(a_t|s_t).\end{aligned}$$

$\frac{\partial}{\partial \theta} \log p_\theta(\tau)$ 是和状态转移概率无关，只和策略函数相关。

策略梯度

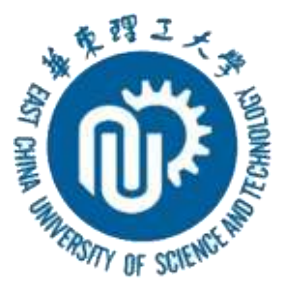


因此，策略梯度 $\frac{\partial \mathcal{J}(\theta)}{\partial \theta}$ 可写为

$$\begin{aligned}\frac{\partial \mathcal{J}(\theta)}{\partial \theta} &= \mathbb{E}_{\tau \sim p_{\theta}(\tau)} \left[\left(\sum_{t=0}^{T-1} \frac{\partial}{\partial \theta} \log \pi_{\theta}(a_t | s_t) \right) G(\tau) \right] \\ &= \mathbb{E}_{\tau \sim p_{\theta}(\tau)} \left[\left(\sum_{t=0}^{T-1} \frac{\partial}{\partial \theta} \log \pi_{\theta}(a_t | s_t) \right) \left(G(\tau_{1:t-1}) + \gamma^t G(\tau_{t:T}) \right) \right] \\ &= \mathbb{E}_{\tau \sim p_{\theta}(\tau)} \left[\sum_{t=0}^{T-1} \left(\frac{\partial}{\partial \theta} \log \pi_{\theta}(a_t | s_t) \gamma^t G(\tau_{t:T}) \right) \right],\end{aligned}$$

其中 $G(\tau_{t:T})$ 为从时刻 t 作为起始时刻收到总回报

$$G(\tau_{t:T}) = \sum_{t'=t}^{T-1} \gamma^{t'-t} r_{t'+1}.$$

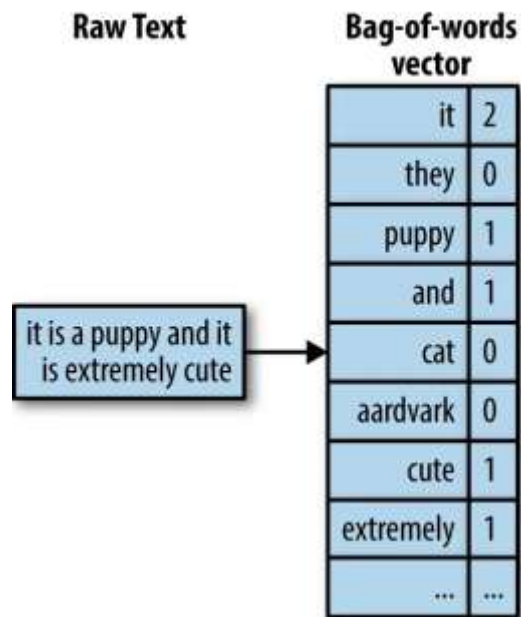


大模型基础技术



一、语言模型发展历程

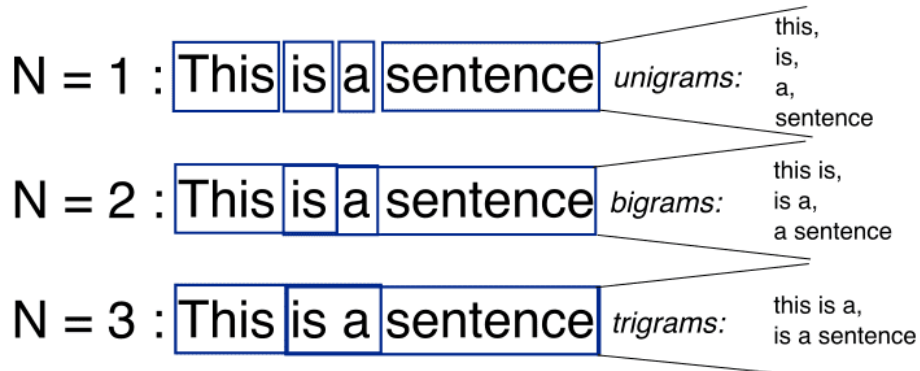
➤ 语言模型的发展历程中的重要概念/模型：bag of words, n-gram,



1. 分词 (Tokenization) : 在训练集文档上划分出单个词或词元的词典。
2. 统计: 统计每个词或词元出现的频率。
3. 向量化 (Vectorization) : 将输入文档表示成所有词或词元的频率向量。

缺陷: 单个词包含的信息量太少

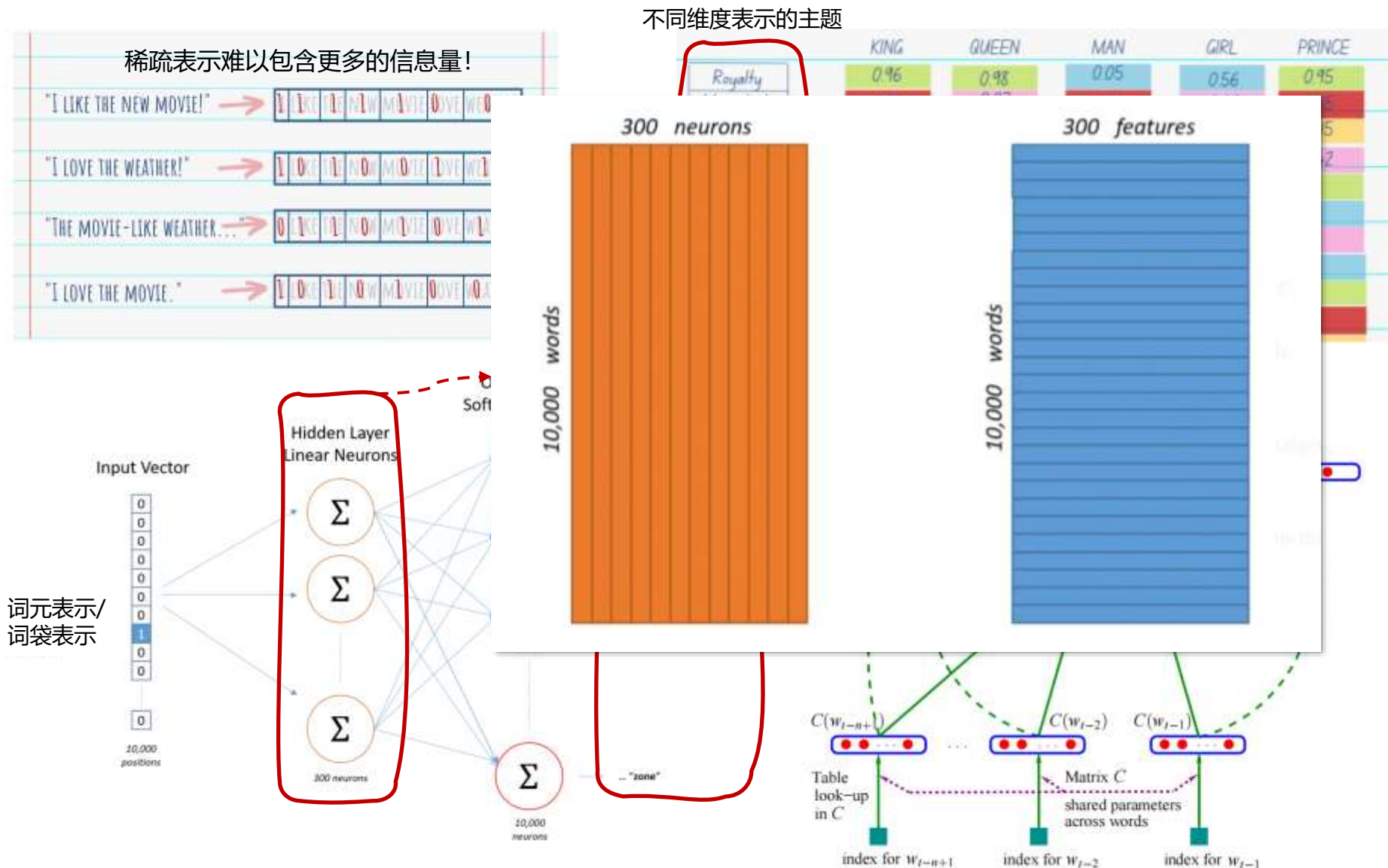
n-gram



随着n数量的增加使得计算量爆炸 💥

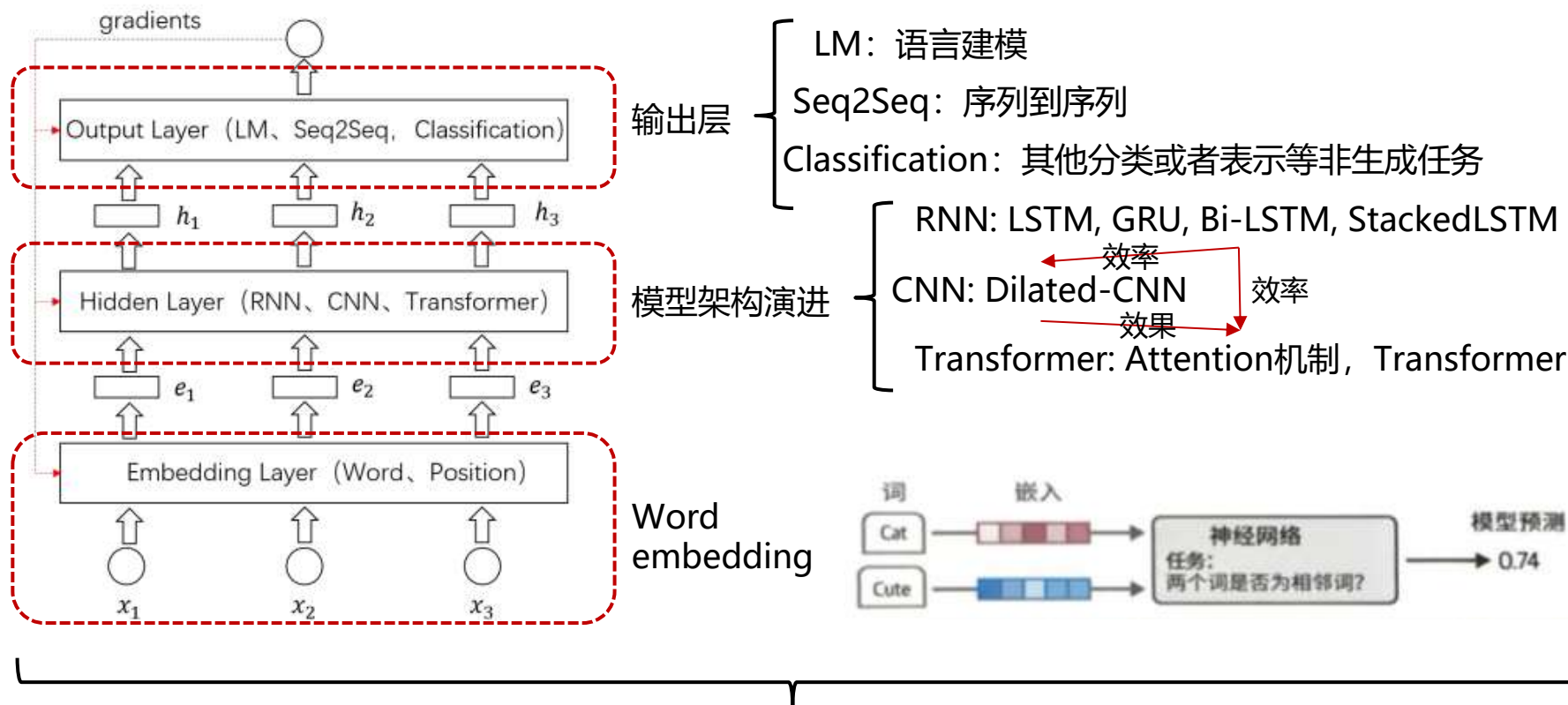
一、语言模型发展历程

➤ 语言模型发展历程中的重要模型/概念：神经网络表示建模——word2vec



一、语言模型发展历程

➤ 语言模型发展历程中的重要模型/概念：语言模型基本架构

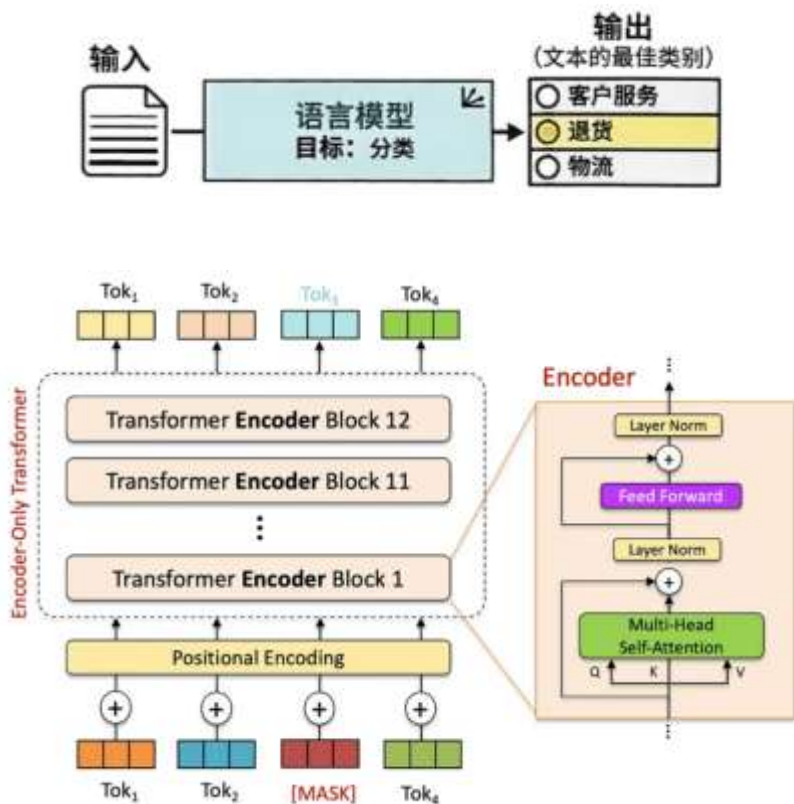


预训练+后训练、提示工程

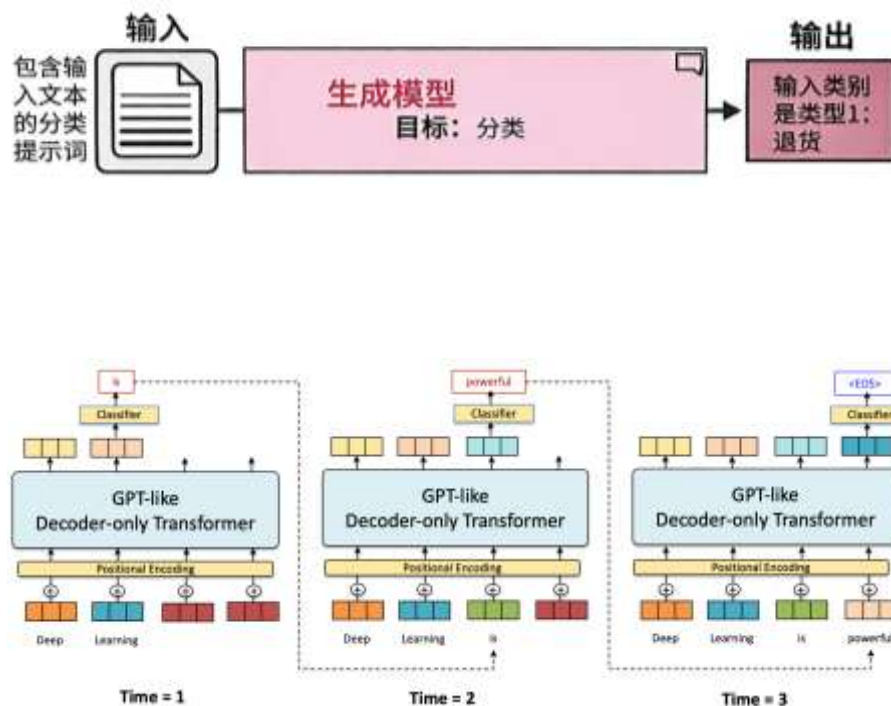
二、NLP基础概念

基础概念与技术：表示模型与生成模型

➤ 表示模型：仅使用Transformer-编码器，代表模型BERT



➤ 生成模型：仅使用Transformer-解码器，代表模型GPT



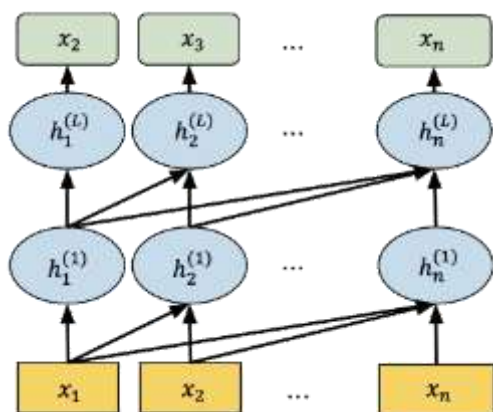
三、Transformer预训练

➤ Transformer的预训练：预训练模型的基础思路

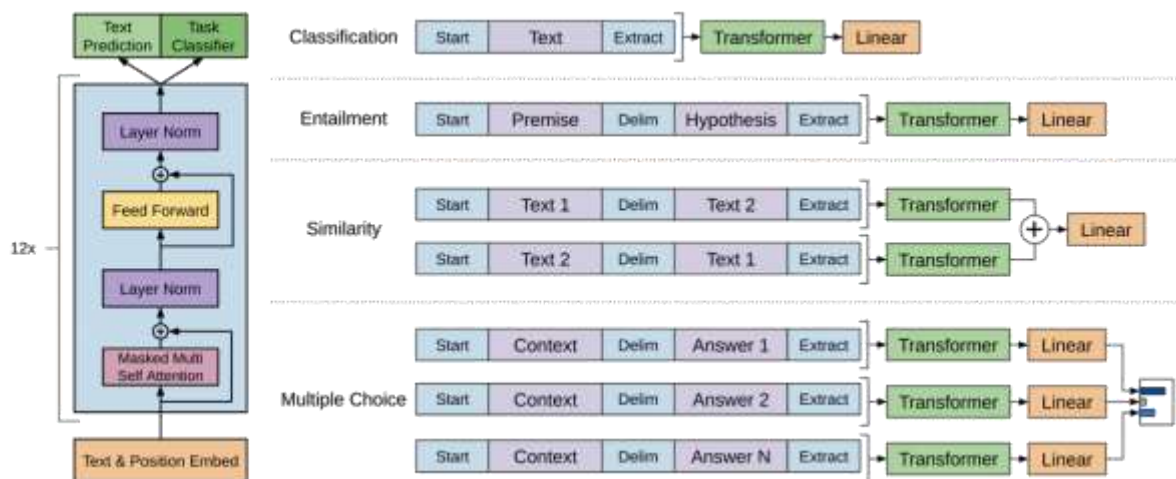
？ 如何把预训练应用到 Transformer上；

？ 如何保持 Embedding 和 Hidden 层结构不变把 Transformer 应用到不同的 NLP 任务上；

OpenAI 提出了一种预训练框架 Generative Pre-Training (GPT)；GPT提出用与 Vanilla Transformer 类似的 LM 任务，即对于每一句话预测每个位置的下一个单词。



注： $h_n^{(L)}$ 代表一个Transformer Block.



在下游NLP任务上微调方法

三、Transformer预训练

➤ Transformer的预训练：预训练模型的基础思路

? 如何把预

? 如何保持

NLP任务

Google提出

把预训练的

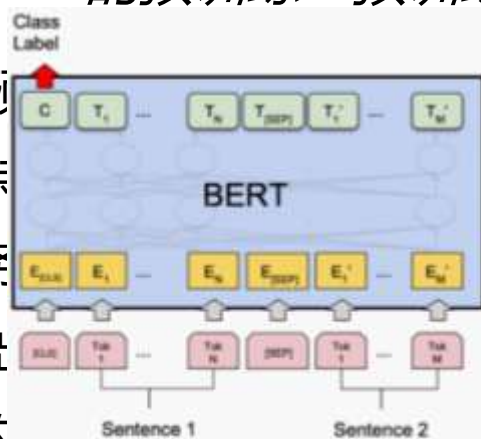
ner 应用到不同的

odeling (BERT),

(MLM) 任务。

Masked Language
Modeling (MLM)

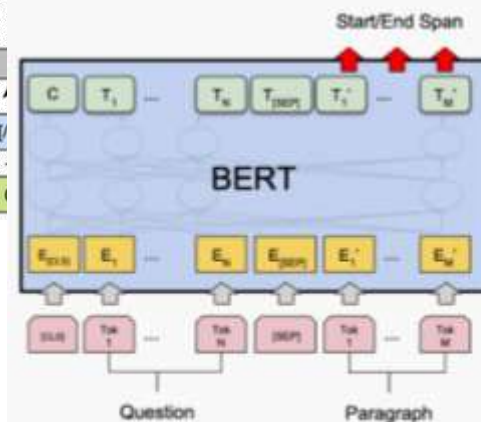
Token
embeddings
Position
embeddings



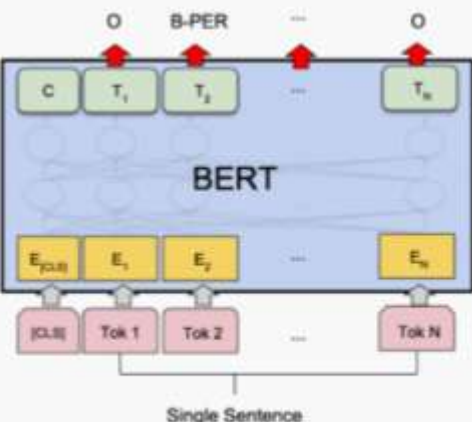
(a) Sentence Pair Classification Tasks:
MNLI, QQP, QNLI, STS-B, MRPC,
RTE, SWAG



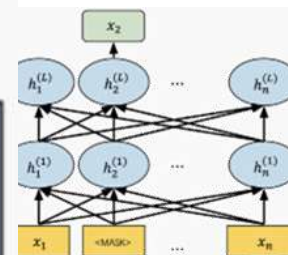
(b) Single Sentence Classification Tasks:
SST-2, CoLA



(c) Question Answering Tasks:
SQuAD v1.1

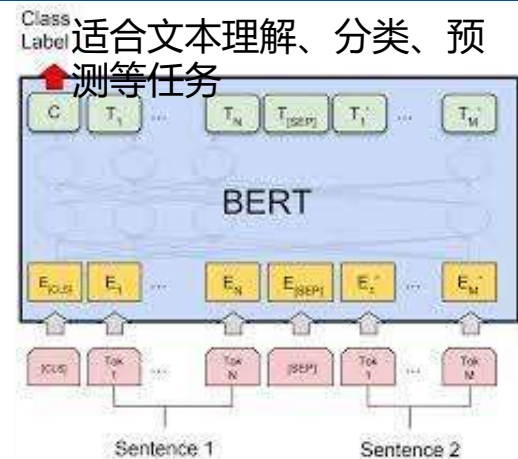


(d) Single Sentence Tagging Tasks:
CoNLL-2003 NER

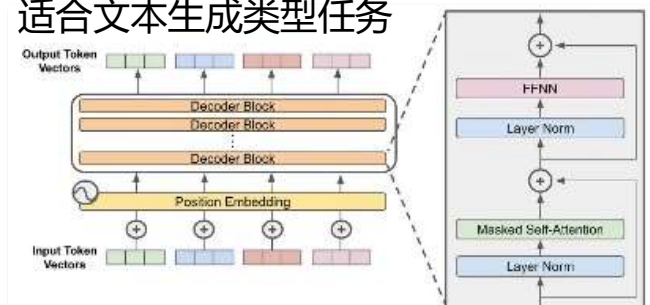


四、预训练语言模型

- **Encoder-only流派**: 仅包含Transformer编码器 (encoder-only), 采用了双向自注意力机制, 模型在处理输入序列时能**考虑上下文的所有 token**。——代表模型: Bert, Roberta。
- **Decoder-only流派**: 仅包含Transformer解码器 (decoder-only), 采用了掩码自注意力机制, 模型执行自回归 (auto-regressive) 形式的预测, 在计算**每个 token 的注意力时后续位置是被“遮罩”起来的**、只能使用前面位置的信息。——代表模型OpenAI 的GPT系列, Meta的OPT系列。
- **Encoder-Decoder流派**: 采用序列到序列形式, 同时包含了编码器和解码器模块 (Encoder-Decoder) ——代表模型Google的T5。



适合文本生成类型任务



适合翻译、对话类型任务

